

Lean from First Principles

Dependent Types, the Calculus of Constructions, and the Architecture of Leant

A beginner's path from “a term has a type” to verified type-directed synthesis

Repository snapshot

Leant	49c8f9f100a6 (main, inspected 16 August 2026)
Djex used by Leant	7d65eba6f753 (the lib/Djex submodule)
Djex standalone	0cf82f2aa22d (main, inspected for the current shared architecture)
Lean 4	79c4cd09fdab (master)

Preface

This guide has a deliberately narrow destination and a deliberately broad runway. The destination is practical: after reading it, an absolute beginner should be able to open the Leant and Djex repositories, understand what problem `:synth` is trying to solve, follow the major data transformations, and judge correctly what a successful candidate or a negative result means. The runway is broad because those facts make little sense until Lean’s central idea is internalized: *types may depend on values, propositions are types, and proofs are ordinary typed terms checked by a small kernel*.

The guide is not a general catalogue of Lean tactics, a mathlib tutorial, or a replacement for the official Lean books. It concentrates on the pieces that determine Leant’s design:

- the Curry–Howard correspondence and type inhabitation;
- dependent function types, dependent pairs, universes, inductive families, equality, and transport;
- the Calculus of Constructions and the inductive extension used by Lean;
- the separation between surface syntax, elaboration, metaprogramming, and kernel expressions;
- the gap between Lean’s rich dependent type theory and Djex’s deliberately smaller Haskell-like synthesis language;
- the translation, search, rendering, verification, and evidence boundaries in Leant.

The existing LaTeX documents in the Leant repository use two useful idioms. The user manual in `docs/Leant.tex` uses LuaLaTeX and Unicode-safe transcript boxes; the longer architecture reports use chapters, pinned revisions, source trails, diagrams, and explicit trust-boundary boxes. This document combines those idioms. All Lean snippets are presented as Unicode-capable verbatim material, and source-specific statements are tied to the pinned repository state above.

The one-sentence thesis

Lean provides the rich language and the final type checker; Leant projects selected Lean goals into a smaller synthesis problem for Djex, turns Djex’s answers back into Lean syntax, and asks Lean to accept or reject every displayed candidate.

How to read this guide

For the shortest route to the Leant implementation, read Chapters 1–4, then Chapters 6–11, and finally Parts III and IV. The formal-rule boxes can be skipped on a first pass. They are there to remove ambiguity when one later reads kernel code or reasons about completeness.

A few notational conventions recur:

- $\Gamma \vdash t : A$ means that in context Γ , term t has type A .
- $A \rightarrow B$ is a non-dependent function type.
- $\Pi(x : A), B(x)$ is a dependent function type; Lean writes it as $(x : A) \rightarrow B$ or $\forall x : A, B$.
- $t \equiv u$ means *definitionally equal*: Lean can see the equality by computation and built-in conversion rules, without an explicit equality proof.
- “kernel” always means the trusted checker for fully elaborated terms, not the parser, elaborator, tactic engine, or synthesis engine.

Contents

Preface	i
I The Core Idea	1
1 A First Encounter with Lean	2
1.1 Lean is both a language and a checker	2
1.2 Declarations: <code>def</code> , <code>theorem</code> , <code>example</code> , and <code>opaque</code>	2
1.3 Terms, types, and types of types	3
1.4 What Lean adds	3
2 Reading Lean Syntax Without Fear	5
2.1 Function types and function values	5
2.2 Named parameters and binder styles	5
2.3 Definitions by equations and pattern matching	6
2.4 Products, structures, and constructor notation	6
2.5 Unicode and ASCII	7
2.6 Term mode and tactic mode	7
3 Propositions Are Types, Proofs Are Terms	9
3.1 The Curry–Howard dictionary	9
3.2 Implication is a function	9
3.3 Conjunction is product-like	9
3.4 Disjunction is sum-like	10
3.5 Truth, falsity, and negation	10
3.6 Universal and existential quantification	10
3.7 Constructive logic and classical additions	11
4 Type Inhabitation Is Program Synthesis	12
4.1 The question asked by a type	12
4.2 A type constrains wiring, not behavior completely	12
4.3 Soundness and completeness are different promises	13
4.4 Search as proof construction	13
II Dependent Type Theory	15
5 Judgments, Contexts, and Substitution	16

5.1	The typing judgment	16
5.2	Variable and weakening rules	16
5.3	Substitution is the engine of dependency	16
5.4	Binding, alpha-equivalence, and capture avoidance	17
5.5	Metavariables are temporary unknowns	17
6	Dependent Function Types: The Pi-Type	19
6.1	From arrows to dependent arrows	19
6.2	Formation, introduction, and elimination	19
6.3	Four uses of the same constructor	20
6.3.1	Functions from values to values	20
6.3.2	Functions from types to values	20
6.3.3	Functions from values to types	20
6.3.4	Functions from types to types	20
6.4	Universal propositions are dependent functions	20
6.5	Dependency can be computationally relevant	21
6.6	Rank and prenex quantification	21
7	Dependent Pairs, Subtypes, and Existence	22
7.1	The Sigma-Type	22
7.2	Why dependent pairs are harder to translate	22
7.3	Subtypes	22
7.4	Existence in Prop is not data-level Sigma	23
7.5	Structures are one-constructor inductives	23
8	Universes, Sort, Type, and Prop	24
8.1	Why types need types	24
8.2	Lean’s universes are non-cumulative	24
8.3	Universe polymorphism	24
8.4	The meaning of <code>imax</code>	25
8.5	Proof irrelevance	25
8.6	Large elimination and restrictions	25
9	Computation and Definitional Equality	27
9.1	Two ways terms can be equal	27
9.2	The main reduction rules	27
9.2.1	Beta reduction	27
9.2.2	Delta reduction	27
9.2.3	Iota reduction	27
9.2.4	Zeta reduction	28

9.2.5	Projection reduction	28
9.3	Weak-head normalization	28
9.4	Conversion in typing	28
9.5	Opaque declarations and theorems	28
9.6	Why a renderer cannot implement definitional equality itself	28
10	Inductive Types, Recursors, and Termination	30
10.1	Data by constructors	30
10.2	Parameters and indices	30
10.3	Recursors are the primitive eliminators	30
10.4	Pattern matching is elaborated	31
10.5	Strict positivity	31
10.6	Termination and totality	31
10.7	Why recursive synthesis is a separate problem	31
11	Equality, Rewriting, and Transport	33
11.1	Equality is an indexed inductive family	33
11.2	Reflexivity and computation	33
11.3	Transport	33
11.4	Transport in indexed families	33
11.5	Dependent pattern matching creates equalities implicitly	34
11.6	Homogeneous and heterogeneous equality	34
12	The Calculus of Constructions and Its Inductive Extension	35
12.1	Why the name matters	35
12.2	The lambda cube	35
12.3	What inductive types add	36
12.4	A compact core grammar	36
12.5	Normalization, consistency, and total programs	36
12.6	Why Djinn does not search CoC directly	36
III	How Lean Implements the Theory	38
13	From Surface Syntax to a Kernel Term	39
13.1	Lean source is intentionally incomplete	39
13.2	The pipeline	39
13.3	Expected-type-directed elaboration	39
13.4	Unification and metavariable assignment	40
13.5	Implicit arguments and placeholders	40
13.6	Type classes are ordinary dependent parameters plus search	40

13.7	Coercions and overloaded notation	41
13.8	Auto-implicit names	41
14	The Kernel Representation: Level, Expr, and Declaration	42
14.1	Universe levels	42
14.2	The expression datatype	42
14.3	One constructor for arrows and quantifiers	42
14.4	Applications are spines	43
14.5	Bound and free variables	43
14.6	Declarations and environments	43
14.7	Inductive metadata	44
15	Trust, Tactics, Metaprograms, and Execution	45
15.1	The small-kernel architecture	45
15.2	Tactics are metaprograms over goals	45
15.3	Metaprogramming inside Lean	45
15.4	Axioms and opaque constants	46
15.5	Proof irrelevance and erasure	46
15.6	Safe, unsafe, and partial definitions	46
15.7	External solvers and replay	46
IV	Leant and Djex	48
16	Djex’s Deliberately Smaller Type World	49
16.1	Why an intermediate language exists	49
16.2	Djex’s shared source type	49
16.3	Flexible variables and rigid variables	49
16.4	Djinn’s propositional formula language	50
16.5	Opaque atoms preserve identity, not internal logic	50
16.6	The revision boundary	50
17	The End-to-End Leant Pipeline	52
17.1	A GHCi-shaped frontend for a Lean worker	52
17.2	The major stages	52
17.3	Why translation happens inside Lean	53
17.4	The serializer output is treated as an untrusted protocol	53
17.5	Semantic origin survives textual operations	54
17.6	Backend verification usually dominates latency	54
18	The Leant Synthesis Fragment	55

18.1	A frontend-owned semantic tree	55
18.2	Recognized logical structure	55
18.3	Non-dependent versus dependent forall	56
18.4	Type applications retain arguments when they are type-level	56
18.5	Opaque dependent cargo	56
18.6	Refutation-safety bits	57
18.7	Depth and resource limits are represented, not forgotten	57
19	Inductives, Recursion, Providers, and Instances	58
19.1	Expanding an inductive as a sum of products	58
19.2	Why indexed and dependent-field inductives remain opaque	58
19.3	Recursive types and one-layer structure	58
19.4	Providers turn the environment into premises	59
19.5	Curated library mode	59
19.6	Provider schemes and exact instantiations	59
19.7	Why instance search must be bounded and fail closed	60
19.8	Premise staging	60
20	Djinn and Exference: Two Different Search Contracts	61
20.1	Why Leant has two engines	61
20.2	Djinn: proof search in LJ _T	61
20.3	What Djinn can decide	61
20.4	Exference: best-first typed expression search	62
20.5	Typed candidate graphs	62
20.6	Result merging	62
21	Rendering Candidates Back into Lean	64
21.1	A proof term is not yet Lean source	64
21.2	Universe-agnostic constructor syntax	64
21.3	Binder reconstruction	64
21.4	One semantic candidate, several strings	65
21.5	Name restoration and hygiene	65
21.6	Rendering failure is ordinary candidate failure	65
22	Verification and the Positive Soundness Boundary	67
22.1	The exact verification program	67
22.2	Lazy quota-aware checking	67
22.3	What positive verification guarantees	67
22.4	What positive verification does not guarantee	68
22.5	Binding accepted candidates into the session	68

23 Negative Evidence and the Completeness Ledger	69
23.1 Why “no term found” has several meanings	69
23.2 The chain required for a strong negative result	69
23.3 Exact, safe opacity versus unsafe opacity	70
23.4 Quantifier plans	70
23.5 Provider-relative completeness	70
23.6 Recursive and instance-related downgrades	71
23.7 Djinn versus Exference wording	71
23.8 Failure transparency	71
24 The Interactive Layer: :synth, :prove, Sessions, and Debugging	72
24.1 Expression-oriented interaction	72
24.2 :synth	72
24.3 :prove	72
24.4 Provider caches and environment identity	73
24.5 Debugging a missing candidate	73
24.6 Reading source ownership	73
V Worked Traces and Boundaries	74
25 Worked Structural Synthesis Traces	75
25.1 Composition	75
25.2 Swapping a product	76
25.3 Sum reassociation	76
25.4 A local continuation pattern	76
26 A Complete Negative Trace	78
26.1 The target	78
26.2 Rigid quantification	78
26.3 Exhaustive LJT failure	78
26.4 Why this failure can be conclusive	78
26.5 What would change the conclusion	79
27 Dependent Cargo: What Works Without Dependent Search	80
27.1 Moving an opaque dependent formula	80
27.2 Instantiating the dependent premise does not work by mere opacity	80
27.3 Transport is likewise invisible	81
27.4 Use proof mode to expose structural subgoals	81
28 Polymorphism, Rank-N Use, and Rendering Variants	82

28.1	A polymorphic argument	82
28.2	How the type crosses the boundary	82
28.3	Why a renderer may need variants	82
28.4	Ambiguity weakens negative evidence	83
29	Provider-Guided and Library-Guided Synthesis	84
29.1	A global conversion function	84
29.2	Polymorphic providers	84
29.3	Class-constrained providers	84
29.4	Library recursion as a provider	85
29.5	Why results depend on configuration	85
30	The Optional Length/Z3 Behavioral Layer	86
30.1	Why types are not enough	86
30.2	The authority order	86
30.3	Sealed contracts and typed graphs	86
30.4	Symbolic length interpretation	87
30.5	SMT solving and replay	87
30.6	What the layer can and cannot say	87
31	A Reliable Mental Model for Using and Extending Leant	88
31.1	Ask four questions about every result	88
31.2	Classify goals before searching	88
31.3	Extend at the correct layer	88
31.4	Prefer fail-closed boundaries	89
31.5	The compact architectural picture	89
	Conclusion	90
A	Lean Syntax and Logic Cheat Sheet	91
A.1	Declarations	91
A.2	Binders	91
A.3	Term forms	91
A.4	Logical forms	92
A.5	Inspection commands useful around Leant	92
B	Compact Formal Rules	93
B.1	Contexts and sorts	93
B.2	Dependent products	93
B.3	Local definitions	94
B.4	Conversion	94

B.5	Dependent pairs	94
B.6	Equality elimination	94
B.7	Inductive declarations	94
C	Lean-to-Leant Translation Reference	96
D	Interpreting Leant Outcomes	97
E	Glossary	98
F	Pinned Source Reading Guide	100
F.1	Leant at 49c8f9f100a6	100
F.2	Djex	100
F.3	Lean 4 at 79c4cd09fdab	101

Part I

The Core Idea

A First Encounter with Lean

1.1 Lean is both a language and a checker

Lean is a functional programming language, a language for stating mathematical propositions, and an interactive theorem prover. These are not three unrelated subsystems bolted together. They share one typed term language. A natural number expression is a term. A function is a term. A proposition is a type. A proof is a term whose type is that proposition.

Start with ordinary programming:

Lean: data and functions

```
def twice (f : Nat → Nat) (x : Nat) : Nat :=
  f (f x)

#check twice
#eval twice (fun n => n + 1) 40
```

The declaration says that `twice` accepts a function from natural numbers to natural numbers, accepts a natural number, and returns a natural number. The body is checked against that declared type. Evaluation produces 42.

Now look at a theorem written in the same language:

Lean: a proof as a function

```
theorem andSwap (P Q : Prop) : P ∧ Q → Q ∧ P :=
  fun h => (h.right, h.left)
```

The body is again a term checked against a type. This time the type is a proposition. The input `h` contains proofs of both P and Q ; the output pair puts them in the opposite order. Nothing in the kernel says “switch into proof mode now.” It checks one typed term.

Read the punctuation literally

In $P \wedge Q \rightarrow Q \wedge P$, the arrow is right-associative, so the type means “given a proof of $P \wedge Q$, produce a proof of $Q \wedge P$.” The term `fun h => ...` is a lambda expression: a function accepting that proof.

1.2 Declarations: `def`, `theorem`, `example`, and `opaque`

The most common top-level forms differ mainly in how the result is named and how it may compute.

`def`

Introduces a named definition with a body. Lean may unfold it during computation and definitional equality, subject to transparency policy in higher layers.

`theorem`

Introduces a named proof. Its body is checked, but theorem bodies are treated opaquely by kernel reduction after acceptance. This keeps proof checking manageable and expresses that the proof’s type is the public contract.

`example`

Checks a declaration without adding a durable user-facing name. It is ideal for experiments and, notably, for Lean’s

candidate-verification command.

opaque

Introduces a checked constant whose implementation does not unfold during ordinary reduction. It is useful when only a signature should be visible.

axiom

Introduces a constant with a stated type and no body. The kernel checks later reasoning relative to that assumption; it does not prove the axiom.

Leant verifies a rendered candidate by asking Lean to elaborate a noncomputable example of exactly the requested type:

The shape of Leant's verification command

```
set_option autoImplicit true in
noncomputable example : (GOAL) := (CANDIDATE)
```

That small command is architecturally decisive. The candidate is not accepted because Djex says it has the right type, nor because a string renderer believes it is valid. Lean elaborates it in the live environment and the kernel checks the resulting expression.

1.3 Terms, types, and types of types

Lean's `#check` command asks the elaborator to report a term's type:

Lean: inspecting types

```
#check 17
#check Nat
#check Nat → Nat
#check fun x : Nat => x + 1
#check Prop
#check Type
#check Type 1
```

Typical results are conceptually:

Term	Type
17	Nat
Nat	Type
Nat → Nat	Type
fun x : Nat => x + 1	Nat → Nat
Prop	Type
Type	Type 1
Type 1	Type 2

A type is itself a term, classified by a universe. This is the first hint of the Calculus of Constructions: the language does not stop at functions between data values. It can quantify over types, compute types, and form functions whose results are types.

1.4 What Leant adds

Lean already has holes, tactics, simplifiers, type-class search, decision procedures, and powerful proof automation. Leant adds a particular interactive workflow inspired by GHCi and Djinn:

A representative Leant interaction

```
:synth ((A → B → C) → (A → B) → A → C)
```

A natural answer is:

Synthesized Lean term

```
fun f g x => f x (g x)
```

The type determines a wiring diagram. Given $f : A \rightarrow B \rightarrow C$, $g : A \rightarrow B$, and $x : A$, the only direct way to build C is to give f an A and a B ; x supplies the first, and $g x$ supplies the second.

Leant’s job is not merely to print a plausible lambda term. It must:

1. ask Lean to elaborate the requested goal into a precise internal expression;
2. decide which structure can be translated into a synthesis fragment and which must remain opaque;
3. run one or both Djex engines under explicit search policies;
4. reconstruct Lean syntax, including binders, constructors, global names, and implicit arguments erased by the search model;
5. re-elaborate each candidate against the exact original goal;
6. report negative results only with the strength justified by both translation completeness and search completeness.

Why this matters for Leant

The beginner’s temptation is to think of Leant as “Lean plus a clever autocomplete.” A more accurate model is a verified compiler pipeline with a lossy middle language. Candidate soundness comes from checking the trip back into Lean; negative evidence depends on proving that the lossy projection was not lossy for this particular query.

Source trail

The current Leant overview and examples are in <https://github.com/VladimirReshetnikov/Leant/blob/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0/README.md>. The exact candidate command is generated by `src/Leant/Synth/Fragment.hs`; process management is in `src/Leant/Backend.hs`; orchestration and interactive state are in `src/Main.hs`.

Chapter checkpoint

You should now be able to read “a proof of P ” as “a term of type P ,” and “synthesize a program of type T ” as “search for an inhabitant of T .” Lean owns the exact type theory and final check. Leant owns an interactive translation/search/reconstruction pipeline around that checker.

Reading Lean Syntax Without Fear

Lean’s notation is dense because the same core constructs serve programming and proof. This chapter decodes the subset needed throughout the guide and in the Leant source.

2.1 Function types and function values

A function type is written with an arrow:

Function signatures

```
Nat → Nat
String → Nat → Bool
(A → B) → A → B
```

Arrows associate to the right:

$$A \rightarrow B \rightarrow C \text{ means } A \rightarrow (B \rightarrow C).$$

Function application is written by juxtaposition and associates to the left:

$$f\ x\ y \text{ means } (f\ x)\ y.$$

A lambda expression introduces arguments:

Equivalent lambda spellings

```
fun x : Nat => x + 1
fun (x : Nat) => x + 1
fun x y => x + y
```

Multiple arguments are syntactic convenience for nested one-argument functions. This is *currying*:

$$\lambda x. \lambda y. t$$

has type $A \rightarrow B \rightarrow C$ when $x : A$, $y : B$, and $t : C$.

2.2 Named parameters and binder styles

Lean has four binder styles that matter both to users and to Leant’s renderer:

Surface form	Binder information	Normal use
$(x : A)$	explicit	caller supplies an ordinary argument
$\{x : A\}$	implicit	elaborator usually infers the argument
$\{\{x : A\}\}$	strict implicit	inferred only once a later explicit argument is present
$[x : C]$	instance implicit	type-class synthesis supplies evidence

For example:

Implicit and instance arguments

```
def ident {α : Sort u} (x : α) : α := x

class Named (α : Type u) where
  name : α → String

def display {α : Type u} [Named α] (x : α) : String :=
  Named.name x
```

The braces around α say that Lean should infer the type from x . The square brackets say that Lean should search the current local and global instance environment for a `Named  $\alpha$` value.

Why binder visibility survives translation

Djinn’s logical model does not natively distinguish every Lean binder visibility. Lean therefore records whether a quantified type binder was explicit, implicit, or instance-implicit, and its renderer may insert lambdas or `_` placeholders that the search term itself never contained. A single semantic candidate can consequently produce several textual variants, only one of which elaborates.

2.3 Definitions by equations and pattern matching

Lean lets functions be written as pattern-matching equations:

Structural recursion on lists

```
def append : List α → List α → List α
| [], ys => ys
| x :: xs, ys => x :: append xs ys
```

This notation is elaborated into recursors and auxiliary definitions. The kernel does not trust the equation compiler; it checks the generated term.

Constructor patterns expose the data carried by an inductive value:

Pattern matching on a sum

```
def either (f : A → C) (g : B → C) : A ⊕ B → C
| .inl a => f a
| .inr b => g b
```

A match expression gives the same structure inline:

Inline match

```
fun s : A ⊕ B =>
  match s with
  | .inl a => Sum.inr a
  | .inr b => Sum.inl b
```

2.4 Products, structures, and constructor notation

The type $A \times B$ is an ordinary product. A value can be written (a, b) or, when the expected type identifies a one-constructor structure, with anonymous constructor brackets $\langle a, b \rangle$. Fields can be projected by names or indices.

Products and projections

```
def swap (p : A × B) : B × A :=
  (p.2, p.1)
```

The same anonymous constructor notation works for conjunction because `And` is a proposition-valued inductive type with one constructor:

The same shape in Prop

```
theorem andSwap' (P Q : Prop) : P ∧ Q → Q ∧ P :=
  fun h => (h.2, h.1)
```

Lean exploits expected-type elaboration here. Its renderer can use universe-agnostic shapes such as `(a, b)`, and Lean decides whether the target is `Prod`, `And`, `PProd`, or another compatible structure.

2.5 Unicode and ASCII

Lean's standard notation uses symbols such as $\rightarrow$, $\forall$, $\wedge$, $\vee$, $\neg$, $\{$, and $\}$. Editors insert them through abbreviations; ASCII alternatives exist for many constructs. The important point is semantic, not typographic:

Lean	Read as	Core idea
$A \rightarrow B$	function / implication	non-dependent Π
$\forall x : A, P\ x$	for every x	dependent Π
$P \wedge Q$	conjunction	product-like inductive in Prop
$P \vee Q$	disjunction	sum-like inductive in Prop
$\neg P$	not P	definitionally $P \rightarrow \text{False}$
$\exists x, P\ x$	there exists x	inductive existential in Prop

2.6 Term mode and tactic mode

A theorem body can be a direct term:

Direct proof term

```
theorem compose (f : B → C) (g : A → B) : A → C :=
  fun x => f (g x)
```

or a tactic script beginning with by:

Tactic proof producing the same kind of term

```
theorem compose' (f : B → C) (g : A → B) : A → C := by
  intro x
  exact f (g x)
```

Tactics manipulate goals and construct a term behind the scenes. The kernel receives only the final elaborated term. Lean's `:prove` mode participates at the goal-manipulation layer, while `:synth` chiefly asks for a whole inhabitant of a target type.

Chapter checkpoint

Arrows and applications have opposite associativity; binders carry visibility information; pattern matching is compiled; products and conjunctions can share constructor-shaped syntax; tactic scripts are programs that build proof terms.

These facts explain many seemingly fussy steps in Leant's renderer.

Propositions Are Types, Proofs Are Terms

3.1 The Curry–Howard dictionary

The correspondence used by Lean can be summarized as follows:

Logic	Type theory
proposition P	type $P : \text{Prop}$
proof of P	term $p : P$
$P \Rightarrow Q$	function type $P \rightarrow Q$
$P \wedge Q$	type with a proof of P and a proof of Q
$P \vee Q$	type tagged with either a proof of P or a proof of Q
$\top$	inhabited unit-like proposition
$\perp$	empty proposition
$\forall x : A, P(x)$	dependent function returning a proof for every x
$\exists x : A, P(x)$	package containing a witness and a proof
proof normalization	program reduction

This is not merely an analogy used in documentation. The kernel type checks proofs using the same mechanisms used to type check functions.

3.2 Implication is a function

To prove $P \rightarrow Q$, assume a proof of P and construct a proof of Q :

Implication introduction

```
theorem idProof (P : Prop) : P → P :=
  fun p => p
```

To use a proof of $P \rightarrow Q$, apply it to a proof of P :

Implication elimination

```
theorem modusPonens (P Q : Prop) : (P → Q) → P → Q :=
  fun hpq hp => hpq hp
```

The words “introduction” and “elimination” name the two directions of constructor use. Introduction builds a value of a type; elimination consumes one. Djinn’s LJT proof search is largely a disciplined automation of these structural moves.

3.3 Conjunction is product-like

To prove $P \wedge Q$, provide both components. To use it, project or pattern-match:

Conjunction

```
theorem reassoc (P Q R : Prop) :
  (P ∧ Q) ∧ R → P ∧ (Q ∧ R) :=
  fun h => ⟨h.1.1, ⟨h.1.2, h.2⟩⟩
```

The exact type `And P Q` lives in `Prop`; `Prod P Q` would live in a data universe if P and Q were data types. Their logical shapes are similar, but Lean keeps the nominal families distinct.

3.4 Disjunction is sum-like

A proof of $P \vee Q$ contains a tag saying which side holds. Introduction chooses a tag; elimination handles both cases:

Disjunction

```
theorem orSwap (P Q : Prop) : P ∨ Q → Q ∨ P :=
  fun h =>
    match h with
    | Or.inl hp => Or.inr hp
    | Or.inr hq => Or.inl hq
```

This is the same computational shape as swapping `Sum A B`, though proof irrelevance changes what may later be observed from proposition-valued data.

3.5 Truth, falsity, and negation

`True` has a constructor, so it is inhabited. `False` has no constructors, so a proof of it cannot be built in a consistent context. But if a hypothesis already has type `False`, it can be eliminated into any proposition or type:

Explosion

```
theorem absurdFromFalse (P : Prop) : False → P :=
  fun h => False.elim h
```

Negation is defined as implication to falsity:

$$\neg P \text{ is } P \rightarrow \text{False}.$$

So a proof of $\neg P$ is a function that turns any hypothetical proof of P into contradiction.

3.6 Universal and existential quantification

A universal proof is a dependent function:

Universal quantification

```
theorem eqSelf : ∀ n : Nat, n = n :=
  fun n => Eq.refl n
```

An existential proof packages a witness and evidence:

Existential witness

```
theorem existsEven :  $\exists n : \text{Nat}, n \% 2 = 0 :=$ 
  (0, rfl)
```

The witness can affect the type of the second component. That dependency is why quantifiers are not reducible to ordinary simple function and pair types.

3.7 Constructive logic and classical additions

The core proof interpretation is constructive: a proof should carry enough information to construct its conclusion. For example, a proof of $P \vee Q$ identifies a side. The law of excluded middle, $P \vee \neg P$, is not derivable for an arbitrary proposition from constructive rules alone.

Lean’s libraries can add classical principles through named axioms and derived theorems. Once such a constant is in the term, the kernel can check the term relative to it. The distinction matters in Leant:

- Djinn’s LJIT engine searches intuitionistic logic.
- Leant can optionally offer classical premises after a constructive route is exhausted or refuted.
- A candidate using classical principles is still type-correct Lean, but its axiom dependencies differ.

Lean can report the assumptions on which a declaration depends with `#print axioms name`.

Type correctness is relative to the environment

The kernel proves no theorem “from reality.” It checks that a term has a type using the declarations and axioms in its environment. Sound engineering therefore tracks not only whether a term checks, but which opaque constants, axioms, instance declarations, and imported definitions it relies on.

Chapter checkpoint

Logical connectives have data-like introduction and elimination rules. This turns proof search into typed term synthesis. Djinn can automate a propositional structural fragment because that fragment has a finite, well-understood proof calculus; Lean’s full dependent language is richer and requires additional care.

Type Inhabitation Is Program Synthesis

4.1 The question asked by a type

Given a type T , the inhabitation problem asks:

Does there exist a term t such that $t : T$? If so, construct one.

For a proposition, this is theorem proving. For a data type, it is program construction. For a mixed dependent type, it can be both at once.

Some types have an obvious canonical inhabitant:

Canonical inhabitants

```
fun x : A => x

fun f : A → B => fun g : B → C => fun x : A => g (f x)

fun p : A × B => (p.2, p.1)
```

Other types are uninhabited without additional assumptions:

A structurally impossible polymorphic function

```
∀ A B : Type, A → B
```

After the caller chooses A , B , and a value of A , no source of a B is available. In a total, parametric language, no term can manufacture an arbitrary value of an arbitrary type.

4.2 A type constrains wiring, not behavior completely

A type can determine a surprising amount of structure. The type

$$(A \rightarrow B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow A \rightarrow C$$

essentially forces composition. But many types admit several behaviors:

Many inhabitants, different behavior

```
Nat → Nat

List α → List α

Bool → Bool
```

For $\text{List } \alpha \rightarrow \text{List } \alpha$, possible total inhabitants include identity, reverse, constant empty, duplicate each element, drop every element, and many more. Type-directed synthesis alone cannot know which behavior the user intended.

This gives two distinct kinds of specification:

Structural specification

Which inputs, outputs, polymorphic variables, sums, products, and constraints must fit together. This is what Djinn and Exference principally exploit.

Behavioral specification

Equations, invariants, refinement predicates, resource bounds, or examples that distinguish extensionally different inhabitants of the same type.

Current Leant is strongest on structural synthesis. Its optional Length layer is a deliberately narrow example of behavioral assessment: after ordinary Lean type verification, certain list-spine candidates can be interpreted under a sealed length contract and checked through bounded, replayed arithmetic reasoning.

4.3 Soundness and completeness are different promises

Suppose a tool prints a candidate. Candidate *soundness* asks whether the printed term really has the requested Lean type. Leant answers this by re-elaboration and kernel checking.

Suppose instead the tool prints “uninhabited.” That is a *completeness* claim about the search and translation. To justify it, the tool must know that:

- every relevant part of the Lean goal was represented faithfully;
- every allowed premise and constructor was included;
- no hidden type-class dictionary or opaque constant could carry useful data;
- the search procedure was complete for the represented logic;
- no configured budget, depth bound, queue pruning, or finite instantiation plan left unexplored cases.

A verifier can easily reject a bad positive candidate. It cannot rescue an unjustified negative claim, because there is no candidate to check.

The asymmetry that shapes Leant

A lossy translation can still produce sound positive answers if every answer is checked in the original language. The same lossy translation cannot support a sound “no answer exists” conclusion unless the loss is proven irrelevant for that query.

4.4 Search as proof construction

For the propositional fragment, the type’s outer constructor suggests the next proof step:

Goal shape	Introduction move	Resulting work
$A \rightarrow B$	introduce $x : A$	synthesize B in extended context
$A \wedge B$	build a pair	synthesize both A and B
$A \vee B$	choose an injection	synthesize one branch
$\top$	use unit constructor	done
$\perp$	no introduction rule	must derive from context
atomic P	use a hypothesis or premise	apply something whose result is P

Left rules explain how hypotheses can be consumed. For a hypothesis $h : A \wedge B$, split it. For $h : A \vee B$, case-analyze both branches. For $h : A \rightarrow B$, synthesize an A and apply h to obtain B . A focused or contraction-free calculus controls the order so proof search terminates and avoids redundant duplication.

This is the logical heart of Djinn. Exference reaches similar terms by a different, heuristic best-first exploration over typed expression states.

Source trail

Djinn's object language and proof terms are defined in <https://github.com/VladimirReshetnikov/Djex/blob/0cf82f2aa22debb1de2c7aaa13a2abbe190550d8/djinn/src-core/Djinn/Internal/LJTFormula.hs>. The contraction-free LJT search and its explicit completeness/budget distinction are in `djinn/src-core/Djinn/Internal/LJT.hs`. Exference's priority-queue-driven engine is in `exference/src-core/Language/Haskell/Exference/Core/Internal/Exference.hs`.

Chapter checkpoint

Inhabitation unifies theorem proving and program synthesis. Types strongly constrain structure but often underdetermine behavior. Positive soundness can be recovered by checking candidates in Lean; negative completeness requires an exact fragment and exhaustive search.

Part II

Dependent Type Theory

Judgments, Contexts, and Substitution

Dependent type theory is easiest to understand as a small collection of judgments and rules. The notation may look formal, but it merely makes explicit what Lean’s elaborator and kernel are tracking.

5.1 The typing judgment

The central judgment is

$$\Gamma \vdash t : A,$$

read “under assumptions Γ , term t has type A .” The context is an ordered list of declarations such as

$$\Gamma = (A : \text{Type}, B : \text{Type}, f : A \rightarrow B, x : A).$$

From this context we can derive $\Gamma \vdash fx : B$.

Order matters because later types may mention earlier variables. This context is valid:

$$(A : \text{Type}, x : A, P : A \rightarrow \text{Prop}, h : Px).$$

The type of x mentions A ; the type of P mentions A ; the type of h mentions both P and x . Reordering the entries arbitrarily would make names appear before their declarations.

5.2 Variable and weakening rules

If a context contains $x : A$, then x may be used at type A :

Variable

$$\overline{\Gamma, x : A, \Delta \vdash x : A}$$

provided the context is well formed.

A derivation that does not use a new assumption remains valid after that assumption is added. This is weakening. In code, adding a local variable does not invalidate earlier expressions, though dependent references require the context to remain ordered.

5.3 Substitution is the engine of dependency

Suppose a body has a free variable x :

$$\Gamma, x : A \vdash t : B(x).$$

If another term a has type A , substitution replaces x by a everywhere:

$$\Gamma \vdash t[a/x] : B(a).$$

The crucial point is that substitution occurs not only in the term but also in its type. This is what distinguishes dependent application from ordinary function application.

In Lean, if

A dependent function

```
f : (n : Nat) → Vector α n
k : Nat
```

then `f k` has type `Vector α k`. The argument `k` is substituted into the codomain.

5.4 Binding, alpha-equivalence, and capture avoidance

The terms

$$\lambda x : A. x \quad \text{and} \quad \lambda y : A. y$$

mean the same thing. Bound variable names are presentation aids; renaming them consistently does not change the term. This is alpha-equivalence.

A substitution implementation must avoid accidentally capturing free variables. If one substitutes `y` for `x` in `λy. x`, naively obtaining `λy. y`, the formerly free `y` has been captured. The binder must first be renamed, for example:

$$(\lambda z. x)[y/x] = \lambda z. y.$$

Lean’s kernel representation avoids many name-management problems by using de Bruijn indices for bound variables. Djex’s shared source types and proof terms instead carry explicit identities and therefore contain binder freshening, alpha-normalization, rigid/flexible variable distinctions, and capture-avoiding substitution machinery.

Locally nameless Lean expressions

A kernel lambda body represents its bound variable with `Expr.bvar 0`; an outer binder is `bvar 1`, and so on. During traversals, Lean commonly instantiates bound variables with fresh free-variable identifiers in a local context. The names printed to users do not determine binding identity.

5.5 Metavariables are temporary unknowns

During elaboration, Lean may know that a term must exist before it knows the term. It represents such holes with metavariables. A metavariable has a local context and an expected type; tactics and unification assign it later.

For example, elaborating

An inferred argument

```
List.nil
```

creates an unknown element type `α` until the surrounding expected type or later constraints determine it. Similarly, the placeholder `_` asks the elaborator to create and solve a metavariable.

Metavariables are not accepted by the kernel. The source definition of `Lean.Expr` explicitly permits `mvar` because the same expression datatype is used by the elaborator, but the kernel type checker rejects any expression that still contains one.

Why this matters for Leant

Leant's serializer runs inside Lean's metaprogramming layer, where it can instantiate metavariables, weak-head normalize expressions, inspect binder information, ask whether a term is a class application, and query inductive declarations. Its output must describe a closed enough fragment for Haskell to consume. Later, rendered candidates deliberately reintroduce `_` only as surface syntax so Lean can perform a fresh, exact elaboration.

Chapter checkpoint

A typing judgment is always relative to an ordered context. Dependency means substitution changes types as well as terms. Bound names are not identities. Metavariables are elaboration-time obligations, never kernel-level proof holes.

Dependent Function Types: The Pi-Type

6.1 From arrows to dependent arrows

An ordinary function type $A \rightarrow B$ says that every input has the same result type B . A dependent function type permits the result type to mention the input:

$$\Pi(x : A), B(x).$$

Lean uses several surface forms for this one core construction:

Pi-type notation

```
(x : A) → B x
∀ x : A, B x
{x : A} → B x
[x : C] → B x
```

When x does not occur in B , the dependent product degenerates to an ordinary arrow:

$$A \rightarrow B \quad \text{is notation for} \quad \Pi(_, A), B.$$

This is why the kernel needs only one constructor, `Expr.forallE`, for both function types and universal quantification.

6.2 Formation, introduction, and elimination

The formation rule says that if A is a type and $B(x)$ is a type for each $x : A$, then the dependent function type is a type:

Π formation

$$\frac{\Gamma \vdash A : \text{Sort } u \quad \Gamma, x : A \vdash B : \text{Sort } v}{\Gamma \vdash \Pi(x : A), B : \text{Sort}(\text{imax}(u, v))}$$

Introduction is lambda abstraction:

Π introduction

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash (\lambda x : A. t) : \Pi(x : A), B}$$

Elimination is application, with substitution in the result type:

Π elimination

$$\frac{\Gamma \vdash f : \Pi(x : A), B(x) \quad \Gamma \vdash a : A}{\Gamma \vdash fa : B(a)}$$

These three rules simultaneously explain ordinary functions, polymorphism, universal quantification, and type constructors.

6.3 Four uses of the same constructor

6.3.1 Functions from values to values

Ordinary function

```
def successor : Nat → Nat := fun n => n + 1
```

The codomain does not depend on n .

6.3.2 Functions from types to values

Polymorphic identity

```
def ident {α : Sort u} (x : α) : α := x
```

The function quantifies over a type α and then over a value of that type. Its full type is approximately

$$\Pi(\alpha : \text{Sort } u), \alpha \rightarrow \alpha.$$

6.3.3 Functions from values to types

`Vector α` maps a natural number to a type:

A type family

```
#check Vector
-- Vector : Type u → Nat → Type u
```

For fixed α , `Vector α 3` and `Vector α 4` are different types. A type family is simply a function whose codomain is a universe.

6.3.4 Functions from types to types

`List` is a type constructor:

A polymorphic type constructor

```
#check List
-- List : Type u → Type u
```

At the kernel level, this is again an ordinary constant whose type is a Π -type.

6.4 Universal propositions are dependent functions

The proposition

$$\forall n : \mathbb{N}, n = n$$

is represented by a dependent function type whose codomain lies in `Prop`. Its proof is a function returning a proof for each input:

A universal proof

```
theorem reflAll : ∀ n : Nat, n = n :=
  fun n => rfl
```

The notation $\forall$ changes how humans read the type, not the kernel constructor.

6.5 Dependency can be computationally relevant

Consider a function that returns a vector whose length is its input:

Result type depends on the argument

```
def replicateVec (α : Type u) (a : α) :
  (n : Nat) → Vector α n
| 0 => Vector.nil
| n + 1 => Vector.cons a (replicateVec α a n)
```

For each n , the result has a different type. The recursive branch must return exactly `Vector α (n + 1)`, not merely “some vector.” The type checker tracks the index through the computation.

This expressive power is precisely what a simply typed search calculus lacks. Djinn can treat an entire formula such as $\forall n, P\ n$ as an opaque atom and move it around, but it cannot generally synthesize a term by inspecting n , changing a result index, or transporting across an equality.

6.6 Rank and prenex quantification

A type is rank-1 when universal quantifiers appear only at the outermost level. A higher-rank type places a polymorphic function inside an argument:

A rank-2 shape

```
((∀ α : Type, α → α) → Nat)
```

Djex’s shared Haskell-like type representation supports explicit `ForallType` layers and bounded handling of certain higher-rank uses. But the quantifier is still a type-scheme mechanism, not full term dependency. Leant must preserve Lean binder visibility and, for some instantiations, enumerate reconstruction variants.

Do not confuse two meanings of “forall”

In Lean, every function binder is represented by the same dependent Π constructor. In Djex’s source type language, `ForallType` means Haskell-style type-variable quantification, while `FunctionType` means a value-level arrow. Leant’s translation decides how a Lean `forallE` maps across that split.

Chapter checkpoint

The Π -type unifies functions, polymorphism, universal proofs, and type families. Application substitutes the argument into the result type. Lean’s single kernel constructor is richer than Djex’s separate Haskell-style `forall` and arrow constructors, so translation is semantic rather than syntactic.

Dependent Pairs, Subtypes, and Existence

7.1 The Sigma-Type

A dependent pair packages a first component $x : A$ and a second component whose type may depend on x :

$$\Sigma(x : A), B(x).$$

Lean’s general dependent pair is Sigma:

A length-indexed package

```
def SomeVector (α : Type u) :=
  Sigma fun n : Nat => Vector α n
```

A value of `SomeVector α` contains a natural number n and a vector of exactly that length. The second projection’s type depends on the first projection.

Constructing and consuming a Sigma

```
def oneVector (a : α) : SomeVector α :=
  (1, Vector.cons a Vector.nil)

def packedLength (v : SomeVector α) : Nat :=
  v.1
```

The ordinary product $A \times B$ is the non-dependent special case where B ignores the first component.

7.2 Why dependent pairs are harder to translate

A structural synthesis engine can represent a pair as “produce an A and a B .” For a dependent pair it must instead produce an $a : A$ and then produce a term of $B(a)$. The second search problem is not even known until the first term is chosen. If $B(a)$ contains computation, equality, or indexed inductives, simple propositional search is insufficient.

Lean therefore expands only carefully characterized product-like inductives with non-dependent fields. A genuinely dependent structure, such as a constructor whose later field type mentions an earlier field, remains opaque to Djinn’s structural logic.

7.3 Subtypes

A subtype packages data together with a proposition about it:

A bounded natural number

```
def SmallNat := {n : Nat // n < 10}

def three : SmallNat :=
  (3, by decide)
```


The first component is computational data. The second is a proof in `Prop`. Because proofs are irrelevant for computation, Lean can erase the proof when generating executable code, while still using it during type checking.

A value of `Fin n` is essentially a natural number together with a proof that it is below n . Its type depends on the bound:

An index carrying a bound

```
#check Fin
-- Fin : Nat → Type
```

This lets an array access API require evidence that an index is in range. The caller cannot pass an arbitrary natural number without also discharging the bound.

7.4 Existence in `Prop` is not data-level `Sigma`

Lean's `Exists` is an inductive proposition:

Existential proposition

```
∃ x : A, P x
```

It resembles a dependent pair, but it lives in `Prop`. Lean's elimination restrictions and proof irrelevance prevent arbitrary extraction of computational data from a proof of existence into a data universe. This preserves the distinction between merely proving that a witness exists and carrying a witness as executable data.

Use `Sigma` when the witness is data to be computed with. Use `Exists` when the package is a proposition and only logical consequences are needed.

Similar notation does not imply interchangeable elimination

Both $\langle w, h \rangle : \exists x, P x$ and $\langle w, v \rangle : \text{Sigma } B$ look like pairs. Their universe and elimination behavior differ. Lean's fragment classifier must retain those nominal and sort distinctions even when the search logic sees a similar product shape.

7.5 Structures are one-constructor inductives

A Lean structure is a convenient declaration of an inductive type with one constructor and named projections:

A dependent structure

```
structure SizedBuffer (α : Type u) where
  size : Nat
  data : Vector α size
```

The field `data` depends on `size`. Lean generates a constructor and projections; the kernel checks the inductive declaration and associated recursor. The compact `Expr.proj` node represents projections in elaborated expressions.

Non-dependent records can often be treated as products. Dependent records need transport-aware elimination and remain outside Lean's ordinary structural expansion.

Chapter checkpoint

A Σ -type is a pair whose second component depends on the first. Ordinary products, subtypes, existentials, and structures are related but not identical. Dependent fields are a sharp boundary for a propositional synthesis engine.

Universes, Sort, Type, and Prop

8.1 Why types need types

If every term has a type, what is the type of `Nat`? It is `Type`. What is the type of `Type`? It is `Type 1`. Continuing prevents the inconsistent self-typing rule `Type : Type`.

Lean's primitive notation is `Sort u`. Two common abbreviations are

$$\text{Prop} = \text{Sort } 0, \quad \text{Type } u = \text{Sort}(u + 1).$$

Thus `Type` is `Type 0`, or `Sort 1`.



8.2 Lean's universes are non-cumulative

Lean uses a countable hierarchy of *non-cumulative* universes. A type that inhabits `Type` is not automatically treated as an inhabitant of `Type 1`. Universe-polymorphic definitions are instead instantiated at the required level.

This often surprises users familiar with cumulative systems. The hierarchy still satisfies

Universes themselves inhabit the next universe

```
#check Type
-- Type : Type 1

#check Type 1
-- Type 1 : Type 2
```

but a small data type is not silently lifted into every larger universe. Lean's universe parameters and `max/imax` expressions keep definitions reusable without cumulative coercions.

8.3 Universe polymorphism

A polymorphic identity should work for propositions, ordinary data, and higher-universe objects:

Sort-polymorphic identity

```
def sid.{u} {α : Sort u} (x : α) : α := x
```

The universe parameter `u` is instantiated at each use. Constants store a list of universe parameters in their declaration and a list of universe arguments in each `Expr.const` occurrence.

Lean's internal universe levels are built from:

- `zero`;

- `succ u`;
- `max u v`;
- `imax u v`;
- named parameters;
- elaboration metavariables.

Kernel expressions cannot retain level metavariables, just as they cannot retain term metavariables.

8.4 The meaning of `imax`

The universe of a dependent function type is

$$\text{imax}(u, v) = \begin{cases} 0, & v = 0, \\ \max(u, v), & v > 0. \end{cases}$$

This special operation encodes the impredicativity of `Prop`. If the codomain of a Π -type is a proposition, the whole dependent function type remains a proposition no matter how large the domain is:

$$\Pi(A : \text{Type } 42), P(A) : \text{Prop}.$$

A proposition may quantify over objects from arbitrary universes without leaving `Prop`. By contrast, a data-valued dependent function must live high enough to contain both its domain and codomain structure.

The pinned kernel implementation computes this literally: `infer_pi` collects domain sort levels and folds `mk_imax` from the final codomain level.

8.5 Proof irrelevance

All proofs of the same proposition are treated as definitionally equal by Lean's kernel. If $p, q : P$ with $P : \text{Prop}$, then the kernel's definitional equality can regard p and q as equal without comparing their proof structure.

Proof irrelevance supports efficient erasure and prevents programs from branching on which proof of a proposition was supplied. It also explains why proposition-valued structures are not interchangeable with data-valued structures even when their constructor shapes match.

8.6 Large elimination and restrictions

Because `Prop` is impredicative and proof-irrelevant, arbitrary elimination from proposition-valued inductives into data would allow computational results to depend on erased proofs. Lean therefore restricts such elimination, with special cases for subsingleton-like propositions. One may eliminate `False` into any sort, but one may not generally turn an arbitrary existential proof into a data-level witness.

Why this matters for Leant

The Leant serializer records whether the goal itself is proposition-valued or data-valued and preserves distinctions among `And/Prod/PProd`, `Or/Sum/PSum`, and `empty/unit` families. Djinn may search their common logical shape, but the renderer relies on Lean's expected type to recover the correct universe-specific constructor.

Source trail

Universe syntax is defined in <https://github.com/leanprover/lean4/blob/79c4cd09fdab845aa6066bcbb09a663876601027/src/Lean/Level.lean>. The kernel rules for sorts, applications, and Π universes are implemented in `src/kernel/type_checker.cpp`. The official reference explicitly describes Lean's universes as non-cumulative and `Type u` as `Sort (u+1)`.

Chapter checkpoint

Lean avoids `Type : Type` with a non-cumulative universe hierarchy. `Prop` is `Sort 0`; data universes start at `Sort 1`. The Π universe uses `imax`, making `Prop` impredicative. Proof irrelevance is a kernel conversion principle, not an optional theorem.

Computation and Definitional Equality

9.1 Two ways terms can be equal

Lean distinguishes:

Definitional equality

$t \equiv u$: the type checker recognizes the terms as equal by built-in computation and conversion. No proof object is required.

Propositional equality

$h : t = u$: an explicit term inhabits the inductive equality proposition.

For example,

$$(\lambda x. x + 1) 4 \equiv 5$$

by beta reduction and natural-number computation. Therefore `rfl` can prove an equality whose two sides reduce to the same normal form.

Computation makes reflexivity sufficient

```
example : (fun x : Nat => x + 1) 4 = 5 := rfl
```

9.2 The main reduction rules

9.2.1 Beta reduction

Applying a lambda substitutes the argument into the body:

$$(\lambda x : A. t) a \longrightarrow t[a/x].$$

9.2.2 Delta reduction

A transparent definition may unfold to its body:

$$\text{twice } f x \longrightarrow f(fx).$$

The kernel uses lazy delta reduction and reducibility hints to decide an efficient unfolding order. The hints affect performance, not the logical meaning of transparent definitions.

9.2.3 Iota reduction

A recursor or pattern match applied to a constructor reduces to the matching branch. Informally:

$$\text{match inl } a \text{ with } \dots \longrightarrow \text{left branch with } a.$$

9.2.4 Zeta reduction

A `let` expression substitutes its value into the body:

$$\text{let } x := a; t \longrightarrow t[a/x].$$

9.2.5 Projection reduction

Projecting a field from a constructor reduces to that field. Lean also supports eta-like conversion for functions and nonrecursive structures in the kernel’s definitional equality procedure.

9.3 Weak-head normalization

The kernel rarely normalizes every subterm eagerly. It computes to *weak-head normal form*: enough to expose the outer constructor. If a purported function type is hidden behind a definition, `ensure_pi` weak-head normalizes until it sees a Π or reports an error. If a purported sort is hidden, `ensure_sort` does the analogous work.

This strategy matters operationally. Full normalization can be expensive; type checking usually needs only the head shape and selective definitional equality.

9.4 Conversion in typing

A conversion rule permits a term inferred at type A to be used where a definitionally equal type B is expected:

Conversion

$$\frac{\Gamma \vdash t : A \quad A \equiv B \quad \Gamma \vdash B : s}{\Gamma \vdash t : B}$$

Dependency makes this indispensable. A recursive function may produce `Vector α (Nat.succ n)` while the expected type prints as `Vector α (n + 1)`; whether the terms align depends on reduction and the definitions involved.

9.5 Opaque declarations and theorems

Not every named constant unfolds. Opaque definitions and theorem declarations expose their types but not reducible bodies to ordinary kernel conversion. This modularity keeps checking stable and prevents proof terms from expanding uncontrollably.

A term can still use an opaque theorem as a constant. What it cannot do is rely on computing through the theorem’s hidden proof body.

9.6 Why a renderer cannot implement definitional equality itself

Leant’s Haskell side sees strings, fragment trees, Djex types, and candidate graphs. Reimplementing Lean’s full conversion relation there would require reproducing environment lookup, universe instantiation, recursive recursors, projections, proof irrelevance, native reductions, and transparency behavior. That would create a second, drifting authority.

Instead, Leant uses Lean’s own metaprogramming operations during translation and Lean’s own elaborator/kernel during verification. Haskell-side normalization is limited to its own representations and never grants Lean type authority.

Expected-type elaboration is a semantic service

A candidate such as $\langle x, y \rangle$ is intentionally incomplete as a standalone string. Against an exact expected type, Lean selects a constructor, inserts universe parameters and implicit arguments, resolves notation, checks conversion, and produces a kernel expression. Leant treats that elaboration as the authoritative final lowering pass.

Chapter checkpoint

Definitional equality is computation built into type checking. It includes beta, delta, iota, zeta, projections, proof irrelevance, and eta behavior implemented by the kernel's conversion algorithm. Leant delegates this entire relation back to Lean instead of approximating it in Haskell.

Inductive Types, Recursors, and Termination

10.1 Data by constructors

An inductive declaration introduces a type as the least collection closed under specified constructors:

Natural numbers

```
inductive MyNat where
| zero : MyNat
| succ : MyNat → MyNat
```

Every value of `MyNat` is built by a finite constructor tree. This gives both a way to construct values and a complete way to consume them: handle `zero` and handle `succ n` assuming the recursive result for n .

10.2 Parameters and indices

A parameter is uniform across a whole inductive family. An index may vary by constructor:

An indexed vector family

```
inductive Vec (α : Type u) : Nat → Type u where
| nil : Vec α 0
| cons {n : Nat} : α → Vec α n → Vec α (n + 1)
```

Here α is a parameter and the length is an index. The constructors return different members of the family. Pattern matching on a vector therefore refines what is known about its length.

The Lean kernel's `InductiveVal` explicitly records the number of parameters and indices, constructor names, universe parameters, whether the family is recursive, and other metadata used to construct and check recursors.

10.3 Recursors are the primitive eliminators

For each accepted inductive declaration, Lean generates a recursor. For natural numbers, its shape is conceptually:

$$\text{Nat.rec} : \Pi(\text{motive} : \text{Nat} \rightarrow \text{Sort } u), \text{motive } 0 \rightarrow (\Pi n, \text{motive } n \rightarrow \text{motive } (n + 1)) \rightarrow \Pi n, \text{motive } n.$$

The *motive* is the result type family. If it is constant, recursion returns ordinary data. If it depends on the input, the same recursor performs induction or dependent pattern matching.

This one principle unifies:

- defining a numeric function by recursion;
- proving a proposition by induction;
- constructing an indexed result whose type tracks the input;
- eliminating a sum or product by cases.

10.4 Pattern matching is elaborated

User-friendly equations and `match` expressions are compiled into recursors, auxiliary definitions, and equality transports. The equation compiler is not trusted. Its generated term is checked by the kernel against the declared type.

For a dependent match, each branch may have a refined expected type. Consider a vector head operation that requires nonempty length:

Index refinement by pattern matching

```
def Vec.head : Vec α (n + 1) → α
| .cons a _ => a
```

The impossible `nil` branch is absent because `Vec α 0` cannot unify with `Vec α (n + 1)`.

10.5 Strict positivity

Inductive definitions must satisfy positivity conditions. Roughly, the type being defined may occur recursively only in positions that preserve monotonicity. A forbidden negative occurrence would allow paradoxical self-application and destroy consistency.

Lean’s inductive checker validates positivity, universe constraints, constructor result heads, and elimination restrictions before adding the compiled declarations to the environment.

10.6 Termination and totality

Lean’s logical definitions must terminate. Structural recursion is accepted when recursive calls are made on syntactically smaller constructor fields. More general recursion can be compiled through well-founded recursion, requiring a proof that calls decrease according to a well-founded relation.

The kernel itself does not have a primitive “general recursive definition” constructor. The source comments in `src/Lean/Declarat.lean` state that recursive definitions are compiled using recursors and `WellFounded.fix`. Unsafe or partial definitions belong to separate safety classes and cannot be used to prove safe logical declarations as though they were total.

10.7 Why recursive synthesis is a separate problem

A type such as `List α → List α` does not say whether the result should recurse over the list, preserve length, reverse order, or ignore input. Inventing a recursive program requires choosing:

- an eliminator or recursion scheme;
- the argument on which to recurse;
- branch bodies;
- recursive-call arguments;
- termination evidence;
- possibly a dependent motive.

Lean therefore does not pretend that general recursive synthesis follows from propositional inhabitation. It supports bounded constructor introduction for certain recursive types and can expose curated library functions as provider premises. Recursive elimination is restricted and engine-specific; arbitrary recursor invention remains out of scope for ordinary `:synth`.

Constructor inventory is evidence

For a nonrecursive, nonindexed inductive with explicit non-dependent fields, Leant may translate the declaration into a finite sum of products. For a recursive type it records whether the constructor inventory is complete and how recursive occurrences appear. An incomplete or ambiguous inventory may still help produce positive candidates, but it cannot justify a conclusive negative result.

Chapter checkpoint

Inductive declarations generate constructors and recursors. Parameters are uniform; indices vary and enable type refinement. Pattern matching is compiled and checked. Total recursion is reduced to structurally or well-foundedly justified terms. General recursive synthesis is therefore much richer than propositional proof search.

Equality, Rewriting, and Transport

11.1 Equality is an indexed inductive family

Lean’s propositional equality has one constructor:

Conceptual shape of equality

```
Eq.refl : (a :  $\alpha$ )  $\rightarrow$  a = a
```

The family $\text{Eq } a \ b$ is indexed by both endpoints. The constructor exists only when they are the same term, modulo definitional equality.

An equality proof can therefore refine types when eliminated. This is much stronger than a Boolean comparison.

11.2 Reflexivity and computation

The tactic or term `rfl` constructs $\text{Eq.refl } _$. It succeeds when both sides are definitionally equal:

Reflexivity after reduction

```
example (x : Nat) : (fun y => y) x = x := rfl
example : List.length [10, 20] = 2 := rfl
```

When equality is not definitional, one needs a theorem or a derivation using induction, congruence, rewriting, arithmetic, or another proof method.

11.3 Transport

If $h : a = b$ and $P : A \rightarrow \text{Sort } u$, a value of $P(a)$ can be transported to $P(b)$:

$$\text{Eq.rec} : P(a) \rightarrow a = b \rightarrow P(b).$$

Lean’s rewrite notation hides this eliminator:

Transport across type equality

```
def castValue { $\alpha$   $\beta$  : Sort u} (h :  $\alpha$  =  $\beta$ ) (x :  $\alpha$ ) :  $\beta$  :=
  h  $\triangleright$  x
```

The term x cannot be returned directly because its type is α , not β . The equality proof changes the expected type.

11.4 Transport in indexed families

Suppose $v : \text{Vector } \alpha \ n$ and $h : n = m$. Transport yields a vector at index m :

Changing an index with an equality

```
def changeLength {n m : Nat} (h : n = m)
  (v : Vector α n) : Vector α m :=
  h ▸ v
```

A synthesis engine that treats `Vector α n` and `Vector α m` as unrelated opaque atoms will not discover this term. A synthesis engine that simply erases the indices and treats both as `Vector α` may generate unsound terms. Correct dependent synthesis must reason about the equality and insert transport.

11.5 Dependent pattern matching creates equalities implicitly

When matching an indexed constructor, Lean refines the index and may insert transports behind the scenes. The surface branch looks simple because the elaborator constructs the appropriate motive and equality reasoning.

This is a major line between Leant's `:synth` and `:prove` modes. The former can transport an opaque dependent proposition as a whole if the surrounding logic only rearranges it. It does not generally invent index equalities, dependent motives, or transport chains. In proof mode, standard Lean tactics and user guidance can expose and solve those obligations.

11.6 Homogeneous and heterogeneous equality

Ordinary `Eq` compares terms of one common type. Dependent developments sometimes need heterogeneous equality, where the endpoint types differ. Lean provides tools such as `HEq`, but most practical reasoning reduces heterogeneous equalities to ordinary equality plus transport.

The presence of such relations is another warning against flattening dependent applications into nominal strings. Their argument structure and universe instantiations determine what can be rewritten.

Dependency is not decoration

An index in `Vector α n` can govern which constructors exist, which patterns are possible, and what result type a branch must produce. Erasing the index changes the typing problem. Keeping it only as an opaque identity preserves sound transport of whole values but sacrifices the ability to synthesize index-sensitive computations.

Chapter checkpoint

Propositional equality is an indexed family whose eliminator performs transport. `rfl` uses definitional equality; rewriting uses an explicit proof. Indexed synthesis often requires equality generation and transport, which Leant intentionally does not model in its ordinary structural fragment.

The Calculus of Constructions and Its Inductive Extension

12.1 Why the name matters

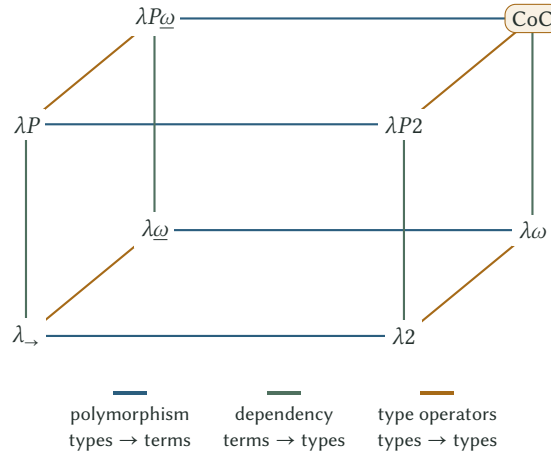
The Calculus of Constructions (CoC) is a compact dependent lambda calculus in which terms, types, and type constructors inhabit one stratified language. It combines:

- ordinary functions from terms to terms;
- polymorphic functions from types to terms;
- type operators from types to types;
- dependent type families from terms to types.

Lean is based on a version of this tradition with a countable hierarchy of non-cumulative universes, an impredicative proof-irrelevant `Prop`, inductive types and recursors, quotient support, and a modern elaboration/programming infrastructure. It is therefore better to say “Lean’s dependent type theory is in the CoC/CIC family” than to identify it mechanically with one historical presentation.

12.2 The lambda cube

The lambda cube classifies typed lambda calculi by three independent forms of dependency added to the simply typed lambda calculus. Let $*$ stand informally for ordinary types and $\square$ for kinds or universes.



The top corner supports all three axes. The ordinary Π -type constructor is expressive enough to encode them uniformly, with universes preventing paradox.

12.3 What inductive types add

Pure CoC has dependent functions but does not by itself provide the practical repertoire of freely generated datatypes and induction principles used in Lean. The Calculus of Inductive Constructions (CIC) adds inductive families together with checked eliminators.

From the user’s perspective, this supplies `Nat`, `List`, equality, conjunction, disjunction, existential quantification, structures, well-founded recursion infrastructure, and essentially every concrete datatype.

From the kernel’s perspective, an inductive block is checked once and compiled into declarations for:

- the type former;
- each constructor;
- recursors and related eliminators;
- no-confusion and projection machinery in higher layers as appropriate.

12.4 A compact core grammar

Ignoring literals, metadata, and projections, the dependent lambda-calculus core can be sketched as

$$t ::= x \mid \text{Sort } u \mid c.\{\bar{u}\} \mid t \, t \mid \lambda(x : A), t \\ \mid \Pi(x : A), B \mid \text{let } x : A := t; u.$$

Lean’s actual `Expr` datatype closely mirrors this grammar, plus free and meta variables, literals, metadata, and compact projections. Inductive types do not require a new general expression constructor: they enter as constants and recursors stored in the environment.

12.5 Normalization, consistency, and total programs

The logical use of Curry–Howard depends on terms not conjuring inhabitants through nontermination. In a language where an infinite loop can be assigned every type, every proposition would become “provable” by divergence. Lean therefore separates safe total definitions from unsafe or partial computation.

Strong normalization is a central metatheoretic property of the pure calculi; Lean’s implemented theory extends the core while preserving a small checkable proof language. The practical guarantee is enforced syntactically and semantically through checked declarations, positivity, universe consistency, and termination compilation.

12.6 Why Djinn does not search CoC directly

Full proof search for arbitrary dependent type theory is qualitatively different from propositional inhabitation. A searcher may need to invent:

- terms that appear inside types;
- motives for dependent eliminators;
- equality proofs and transports;
- universe instantiations;
- recursive definitions and termination proofs;
- instances, coercions, overloaded notation, and environment-dependent constants.

There is no simple finite LJ-style decision procedure for all of that. Leant’s architecture is therefore a principled compromise: use Lean to understand and check the rich language; project an admitted structural fragment to search calculi with known operational properties; preserve unsupported dependent subterms as opaque, alpha-aware atoms; and fail closed on negative evidence.

The architecture follows the theory

Leant is not missing a magical parser flag that would make Djinn “support dependent types.” The search calculus itself is propositional and its proof terms are untyped lambdas with tuple/sum operations. Supporting dependency analytically would require a different proof calculus, candidate representation, unifier, eliminator search, and completeness argument. Opaque cargo is a deliberate semantic boundary, not merely an unfinished pretty printer.

Chapter checkpoint

CoC unifies ordinary functions, polymorphism, type operators, and dependent families. CIC adds inductive families and recursors. Lean belongs to this family but has its own precise universe, proof-irrelevance, quotient, safety, and implementation choices. Djinn searches a much smaller propositional calculus, so Leant must mediate between two genuinely different logics.

Part III

How Lean Implements the Theory

From Surface Syntax to a Kernel Term

13.1 Lean source is intentionally incomplete

Human-friendly Lean code omits information that the kernel requires. Consider:

Compact surface syntax

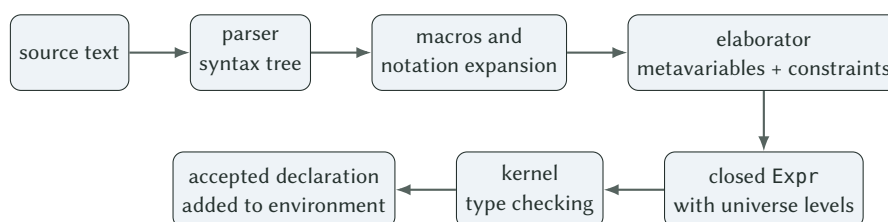
```
fun x => (x, x)
```

The source does not state the type of x , the universe containing that type, the exact product constructor, or its implicit parameters. In a suitable expected context, Lean can infer all of them. Without an expected type, it may report an ambiguity.

This gap is filled by *elaboration*: a constraint-generating and constraint-solving translation from syntax to fully explicit core expressions.

13.2 The pipeline

A simplified Lean command pipeline is:



Tactics, type-class resolution, coercion insertion, notation elaborators, and user metaprograms all operate before the final kernel check. They may be complicated and occasionally buggy without becoming proof authorities: their output is rejected unless it forms a valid kernel term.

13.3 Expected-type-directed elaboration

Elaboration often proceeds bidirectionally:

- *Synthesis mode*: infer a term's type from its syntax.
- *Checking mode*: elaborate a term against a known expected type.

A lambda with an omitted binder type is easy in checking mode:

Expected type supplies the binder

```
example : Nat → Nat := fun x => x
```

The expected $\text{Nat} \rightarrow \text{Nat}$ tells the elaborator that $x : \text{Nat}$ and that the body must produce a natural number.

Anonymous constructor syntax is similarly expected-type dependent:

One syntax, several possible constructors

```
example (p : P) (q : Q) : P ∧ Q := ⟨p, q⟩
example (a : A) (b : B) : A × B := ⟨a, b⟩
```

The string `⟨p, q⟩` does not name `And.intro` or `Prod.mk`. The expected type decides.

13.4 Unification and metavariable assignment

Suppose `id` has type $\{\alpha : \text{Sort } u\} \rightarrow \alpha \rightarrow \alpha$. When elaborating `id 5`, Lean creates unknowns for the universe and α , then unifies the explicit argument's type `Nat` with α . The solved term is conceptually `id.{1} (α := Nat) 5`, though the printed term remains concise.

Dependent unification is more subtle because metavariables may occur inside types. Lean combines higher-order pattern unification, reduction, postponement, and specialized elaboration procedures rather than solving arbitrary higher-order unification, which is undecidable.

13.5 Implicit arguments and placeholders

Implicit parameters are inserted automatically when a constant is applied. A placeholder `_` asks Lean to create a metavariable explicitly. This is useful to Leant's renderer:

A reconstruction placeholder

```
provider _ x
```

The search engine may know that a provider should be applied but may not have retained a surface-level explicit type argument. Emitting `_` delegates that choice to the exact Lean environment. If inference is ambiguous or inconsistent, candidate verification fails.

13.6 Type classes are ordinary dependent parameters plus search

A class declaration defines a structure. An instance is a term of that structure type. A binder `[C α]` is an instance-implicit dependent function argument. The elaborator invokes type-class resolution to synthesize it.

Dictionary passing beneath surface syntax

```
class Render (α : Type u) where
  render : α → String

def showOne {α : Type u} [Render α] (x : α) : String :=
  Render.render x
```

Conceptually, `showOne` receives a dictionary carrying the render operation. That dictionary is a real value and may contain computational content. Erasing it from a synthesis problem is therefore harmless only under carefully restricted conditions.

Hidden dictionaries poison negative evidence

A generated candidate can often leave instance arguments implicit and let Lean resolve them. But a claim that no inhabitant exists cannot ignore a hidden instance premise: the dictionary may provide exactly the operation or value needed to construct the result. Leant records instance binders and refuses strong refutation when their evidence was

not represented exhaustively.

13.7 Coercions and overloaded notation

Numerals, arithmetic operators, field notation, sequence literals, and many other surface forms are overloaded. Their elaboration depends on expected types and instances. A numeral `0` may elaborate as a natural number, integer, rational, finite-ring element, or user-defined type through `OfNat`.

This is another reason candidate verification must compile the exact source text in context. A Haskell renderer cannot decide the meaning of every Lean notation independently of imports and instances.

13.8 Auto-implicit names

With `autoImplicit true`, an undeclared identifier in a declaration header may be introduced as an implicit variable when Lean can infer an appropriate kind. Leant's verification command deliberately enables the option because synthesized goals and user sessions may rely on the same convention.

The option affects elaboration, not kernel semantics. The resulting declaration still contains explicit binders and universe parameters in its core type.

Elaboration success is stronger than parsing, weaker than behavioral correctness

A candidate that parses may still fail to resolve names, infer parameters, solve instances, or satisfy the expected type. A candidate that elaborates and kernel-checks is a genuine inhabitant of the goal. It may nevertheless be the wrong inhabitant for the user's unstated behavioral intent.

Chapter checkpoint

Lean source relies on expected types, metavariables, unification, implicit arguments, type-class search, coercions, and notation elaborators. The output is a fully explicit kernel expression. Leant intentionally emits syntax that Lean must elaborate, because that is the only reliable way to recover exact environment-dependent details.

The Kernel Representation: Level, Expr, and Declaration

14.1 Universe levels

The datatype `Lean.Level` represents universe expressions:

Constructor	Meaning	Typical source form
<code>zero</code>	level 0	<code>Prop</code>
<code>succ u</code>	successor	<code>Type u</code> uses <code>succ u</code>
<code>max u v</code>	maximum	universe of data-valued combinations
<code>imax u v</code>	impredicative maximum	universe of a dependent product
<code>param n</code>	declaration universe parameter	<code>. {u}</code>
<code>mvar id</code>	unsolved elaboration level	temporary only

Level expressions are normalized and compared modulo their algebraic meaning. A closed kernel declaration may mention named parameters but not unresolved level metavariables.

14.2 The expression datatype

At the pinned Lean revision, the core expression datatype has the following constructors:

Constructor	Role	Important detail
<code>bvar</code>	bound variable	de Bruijn index into surrounding binders
<code>fvar</code>	free variable	stable identifier backed by a local-context declaration
<code>mvar</code>	metavariable	elaboration hole; rejected by the kernel
<code>sort</code>	universe term	<code>Prop</code> , <code>Type u</code> , and general <code>Sort u</code>
<code>const</code>	global constant	exact name plus universe arguments
<code>app</code>	application	applications are left-nested binary nodes
<code>lam</code>	lambda abstraction	name hint, domain, body, binder information
<code>forallE</code>	dependent product	represents both Π and non-dependent arrow
<code>letE</code>	local definition	declared type, value, body, and nondependency flag
<code>lit</code>	literal	natural or string literal with kernel reduction support
<code>mdata</code>	metadata wrapper	definitionally equal to the wrapped expression
<code>proj</code>	structure projection	type name, field index, and structure expression

This is strikingly small. There is no kernel node for `if`, `match`, `structure`, `theorem tactics`, `notation`, `list syntax`, or `type classes`. Those are elaborated into applications of constants, lambdas, Π -types, recursors, and projections.

14.3 One constructor for arrows and quantifiers

The expression

A non-dependent arrow

$$\text{Nat} \rightarrow \text{Bool}$$

and the expression

A dependent universal

$$(n : \text{Nat}) \rightarrow \text{Fin } (n + 1)$$

both elaborate to `forallE`. The body of the first does not contain the bound variable; the body of the second does. Surface $\forall$ is also the same node.

This single fact explains much of Leant’s serializer complexity: it must inspect domain sorts, binder visibility, body dependency, application heads, and local variable roles to classify a Lean Π into Djex-compatible type quantification, value implication, opaque dependency, or instance evidence.

14.4 Applications are spines

The term `f a b c` is represented as

$$(((f\ a)\ b)\ c).$$

Utilities recover the head and argument array. Leant uses this view to recognize named logical families such as `And P Q`, nominal type-constructor applications, class heads, and inductive family applications.

A dependent application cannot be classified solely by its printed head. Its arguments may be types, values, indices, proofs, or universe-polymorphic terms. The serializer asks Lean for each argument’s type and whether the whole application is itself type-valued.

14.5 Bound and free variables

Inside stored declarations, lambda and Π bodies use de Bruijn bound variables. Metaprograms often open a binder by creating a fresh free variable, adding its declaration to the local context, and instantiating the body. This makes its type and binder metadata available through ordinary APIs.

Leant’s generated serializer follows that pattern when traversing dependent forall’s and provider schemes. The emitted fragment uses explicit string identities, but those strings are allocated under alpha-aware discipline rather than copied naïvely from pretty-printed binder names.

14.6 Declarations and environments

The global environment maps names to checked declaration records. Major declaration categories include:

- axioms;
- transparent definitions;
- theorems;
- opaque definitions;
- quotient primitives;
- inductive declarations and their compiled constants;

- unsafe, partial, and mutual definitions used under restricted safety rules.

A constant occurrence stores only its name and universe arguments. The environment supplies its type, value if reducible, safety information, and inductive metadata.

This environment dependence is why a bare goal string is not a complete synthesis problem. The same spelling can resolve differently under namespaces and imports; constructors and providers available in one session may not exist in another; instances alter elaboration; reducible definitions change visible structure.

14.7 Inductive metadata

For an inductive family, the environment records parameters, indices, constructors, recursion status, universe parameters, and recursor information. Leant queries this live metadata instead of trying to infer constructors from names or printed declarations.

The distinction among parameter, index, and dependent constructor field is semantic:

- parameters may be carried uniformly into an expanded nominal family;
- indices generally force the family to remain opaque;
- non-dependent explicit fields may become product components;
- recursive occurrences require bounded and inventory-aware handling;
- hidden or dependent fields prevent an exact first-order sum-of-products translation.

Source trail

The exact expression constructors and locally nameless design are documented in <https://github.com/leanprover/lean4/blob/79c4cd09fdab845aa6066bcbb09a663876601027/src/Lean/Expr.lean>. Universe levels are in `src/Lean/Level.lean`; declaration records and inductive metadata are in `src/Lean/Declaration.lean`. Kernel inference and definitional equality are in `src/kernel/type_checker.cpp`.

Chapter checkpoint

Lean’s trusted term language is small: levels, variables, sorts, constants, applications, lambdas, dependent products, lets, literals, metadata, and projections. Rich surface constructs become these nodes plus environment declarations. Leant translates from this exact semantic representation, not from pretty-printed Lean text.

Trust, Tactics, Metaprograms, and Execution

15.1 The small-kernel architecture

Lean’s trust story separates *finding* a proof from *checking* it. The kernel is responsible for:

- checking universe levels and declaration parameters;
- inferring expression types;
- verifying applications and dependent substitution;
- checking definitional equality;
- validating inductive declarations and recursors;
- enforcing safety boundaries on declarations.

The parser, macro system, elaborator, tactic framework, simplifier, compiler, and external automation may all be larger. They are proof-producing or term-producing infrastructure. A bug can cause failure, poor diagnostics, or a rejected term; it should not make an invalid term kernel-valid.

15.2 Tactics are metaprograms over goals

A tactic state contains metavariable goals. A tactic may:

- introduce a binder;
- apply a theorem and create subgoals;
- refine a goal with a partially specified expression;
- normalize or rewrite its target;
- invoke search or decision procedures;
- assign solved metavariables.

When no goals remain, the assigned expression is submitted through the ordinary checking path. Tactics do not add a second notion of proof.

Lean’s `:prove` command can be understood at this level: it interacts with a live proof state, asks its synthesis machinery for suitable terms at selected goals, and returns suggestions that Lean checks in context.

15.3 Metaprogramming inside Lean

Lean exposes APIs for expressions, local contexts, environments, declaration lookup, weak-head normalization, unification, instance synthesis, and syntax construction. Lean uses generated Lean metaprograms to perform tasks that must respect the exact Lean semantics:

- elaborate the raw goal;
- inspect the resulting `Expr`;

- classify logical connectives and Π binders;
- inspect inductive declarations and constructor fields;
- discover provider constants from the live environment;
- serialize a bounded semantic fragment to an S-expression.

The Haskell process does not link against Lean’s internal objects directly. It sends commands to a persistent Lean REPL worker and receives structured JSON responses containing messages, environment identifiers, and results.

15.4 Axioms and opaque constants

An axiom is a trusted assumption because no body is checked. Quotient primitives and choice-related constants are part of Lean’s foundational environment design. Imported projects may add their own axioms.

An opaque theorem is different: its body was checked when declared, but later conversion does not unfold it. Both appear as constants during downstream checking, yet their provenance differs. `#print axioms` follows dependencies to report axiomatic assumptions.

15.5 Proof irrelevance and erasure

Terms whose types lie in `Prop` are computationally irrelevant. The compiler may erase them. This permits specifications to carry proofs without runtime cost, but it constrains elimination from `Prop` into data.

The kernel still type checks proof arguments before erasure. A generated program cannot bypass a required proof merely because it will not exist at runtime.

15.6 Safe, unsafe, and partial definitions

Lean is also a practical programming language. It allows unsafe and partial definitions for executable code, but logical declarations are prevented from depending on unsafe computation as if it were a total proof-producing function. The kernel source checks definition safety when constants are used.

Lean’s candidate-verification command is marked `noncomputable` so candidates may use logically valid noncomputable constants without requesting executable code generation. This relaxes compilation requirements, not type checking, universe checking, or safety checking.

15.7 External solvers and replay

An external solver can be used in several trust arrangements:

1. trust the solver’s yes/no answer directly;
2. demand a certificate and check it in Lean;
3. reconstruct a proof term and ask the kernel to check it;
4. use the solver only as a heuristic or counterexample finder, then independently replay the result.

Lean’s ordinary Djex candidates use arrangement 3 at the final boundary: a Lean term is reconstructed and checked. The optional Length/Z3 layer uses arrangement 4 for its bounded behavioral claims: the solver status alone is not authority; models and positive results are interpreted under a sealed contract and independently replayed by Djex’s semantic layer. Even then, the result is not automatically a general Lean theorem about extensional behavior.

Three different authorities

1. Lean's kernel authorizes "this term has this Lean type."
2. A complete Djinn search over an exact fragment can authorize "this translated formula has no proof."
3. A sealed behavioral checker can authorize only the specific bounded semantic verdict described by its contract and replay rules.

No one authority silently upgrades another.

Chapter checkpoint

Lean's automation constructs terms; the kernel checks them. Metaprograms can inspect the exact live environment without joining the trusted base. Noncomputability, opacity, axioms, unsafe code, and external solvers each have distinct trust meanings. Lean's design is strongest where it preserves those distinctions explicitly.

Part IV

Leant and Djex

Djex’s Deliberately Smaller Type World

16.1 Why an intermediate language exists

Lean’s kernel language is compact, but its types are still fully dependent and environment-sensitive. Searching that language directly would require solving the general problems described in Chapter 12. Djex instead provides synthesis engines for a Haskell-like source type language and, beneath Djinn, a propositional proof calculus.

Leant therefore needs an intermediate semantic boundary. It cannot pass an arbitrary Lean Expr to Djex and hope that both sides mean the same thing. It first classifies the goal into a Leant-owned fragment, then the engine adapter lowers admitted pieces into Djex representations.

16.2 Djex’s shared source type

The current shared datatype `Language.Haskell.Synthesis.Type.Type` has these principal forms:

Djex constructor	Meaning	Lean relationship
<code>TypeVariable</code>	type variable	a selected sort-bound Lean variable
<code>TypeConstructor</code>	nominal type constructor	a Lean constant retained by exact name mapping
<code>TypeApplication</code>	left-associated type application	only admitted type-level applications
<code>FunctionType</code>	non-dependent value arrow	a Lean Π whose body does not depend on the value binder
<code>TupleType</code>	structural boxed/unboxed tuple	product-like fragment
<code>ForallType</code>	type-variable binders, class context, body	selected Lean sort quantifiers and exact provider schemes

This language supports polymorphism, higher-kinded constructor applications, constraints, tuples, and ordinary functions. It has no constructor whose codomain is an arbitrary term-dependent type. A `FunctionType A B` cannot express $\Pi(x : A), B(x)$ when x appears in B .

The shared kind language is correspondingly Haskell-like:

$$\text{ProperTypeKind} \quad | \quad \text{KindVariable } k \quad | \quad \text{FunctionKind } k_1 \ k_2.$$

A sealed checked environment uses ground kinds with no unresolved kind variables. Universe levels are not represented as Lean universes; the frontend must establish that the admitted source type has a compatible kind shape.

16.3 Flexible variables and rigid variables

Djex distinguishes inference variables that unification may instantiate from rigid skolems that must remain fixed:

- a *flexible variable* represents an unknown chosen by the search/unifier;
- a *rigid variable* represents a universally quantified type that search must treat parametrically.

Confusing the two would make polymorphic synthesis unsound. A target such as $\forall A B, A \rightarrow B$ must not be solved by unifying the rigid A and B .

Djex's shared layer also owns capture-avoiding substitution, binder freshening, alpha normalization, validation, constructor-application normalization, and explicit forall-spine processing. These are not cosmetic utilities: they enforce the source calculus on which both engines agree.

16.4 Djinn's propositional formula language

Djinn lowers types further into LJT formulas:

$$F ::= P \mid F \rightarrow F \mid \bigwedge_i F_i \mid \bigvee_i F_i \mid \perp_N.$$

The exact Haskell constructors are `PVar`, implication, n-ary `Conj`, constructor-tagged `Disj`, and nominal `Empty`. A formula atom can be an ordinary symbol or an `OpaqueTypeSymbol` carrying an exact shared source type plus an alpha-normal key.

The proof-term language contains:

- variables, lambdas, and applications;
- tuple construction and splitting;
- sum injection and case analysis;
- constructor descriptors carrying names and arities.

It is deliberately not a dependent proof-term language. Formula atoms have no eliminators except those supplied by assumptions whose formulas expose implications, products, or sums.

16.5 Opaque atoms preserve identity, not internal logic

Suppose Leant encounters the Lean proposition $\forall n : \text{Nat}, P\ n$. If the surrounding goal only needs to move this proposition from one product component to another, Djinn need not understand the quantifier. It can represent the whole type as one atom X and prove

$$X \wedge Q \rightarrow Q \wedge X.$$

The atom must still have stable semantic identity. Comparing pretty-printed strings is unsafe because alpha-equivalent binders may print differently, distinct names may print similarly under namespaces, and renderer choices are not logical keys. Djex's opaque symbol stores a structural source type and compares its alpha-normal key.

Opaque means indivisible, not forgotten

An opaque atom retains enough identity to recognize the same type again and to reconstruct the original Lean spelling. Search may copy it, pass it to a premise, return it, or place it in a product. Search may not inspect its quantifiers, indices, equalities, or constructors.

16.6 The revision boundary

The Leant snapshot studied here vendors Djex commit 7d65eba6f753 as a submodule. The standalone Djex repository was also inspected at 0cf82f2aa22d to understand the current shared architecture and documentation. Statements about Leant's actual executable behavior should be read against the vendored revision; later standalone changes do not automatically enter Leant until the submodule pointer advances.

Source trail

The shared type and kind representations are in <https://github.com/VladimirReshetnikov/Djex/blob/0cf82f2aa22debb1de2c7aaa13a2abbe190550d8/synthesis/src/Language/Haskell/Synthesis/Type.hs> and `synthesis/src/Language/Haskell/Synthesis/Kind.hs`. Djinn formulas and proof terms are in `djinn/src-core/Djinn/Internal/LJTFormula.hs`.

Chapter checkpoint

Djex does not contain Lean's dependent type theory. Its shared source language is polymorphic and Haskell-like; Djinn's core is propositional. Opaque atoms preserve exact alpha-aware identity while withholding internal structure. Leant must prove, query by query, which Lean structure can safely cross this boundary.

The End-to-End Leant Pipeline

17.1 A GHCi-shaped frontend for a Lean worker

Leant is a Haskell executable that presents a persistent interactive session. It feels like a REPL, but the authoritative Lean state lives in a separate Lean process. Leant launches the community Lean REPL through a Lake environment and communicates over a blank-line-delimited JSON protocol.

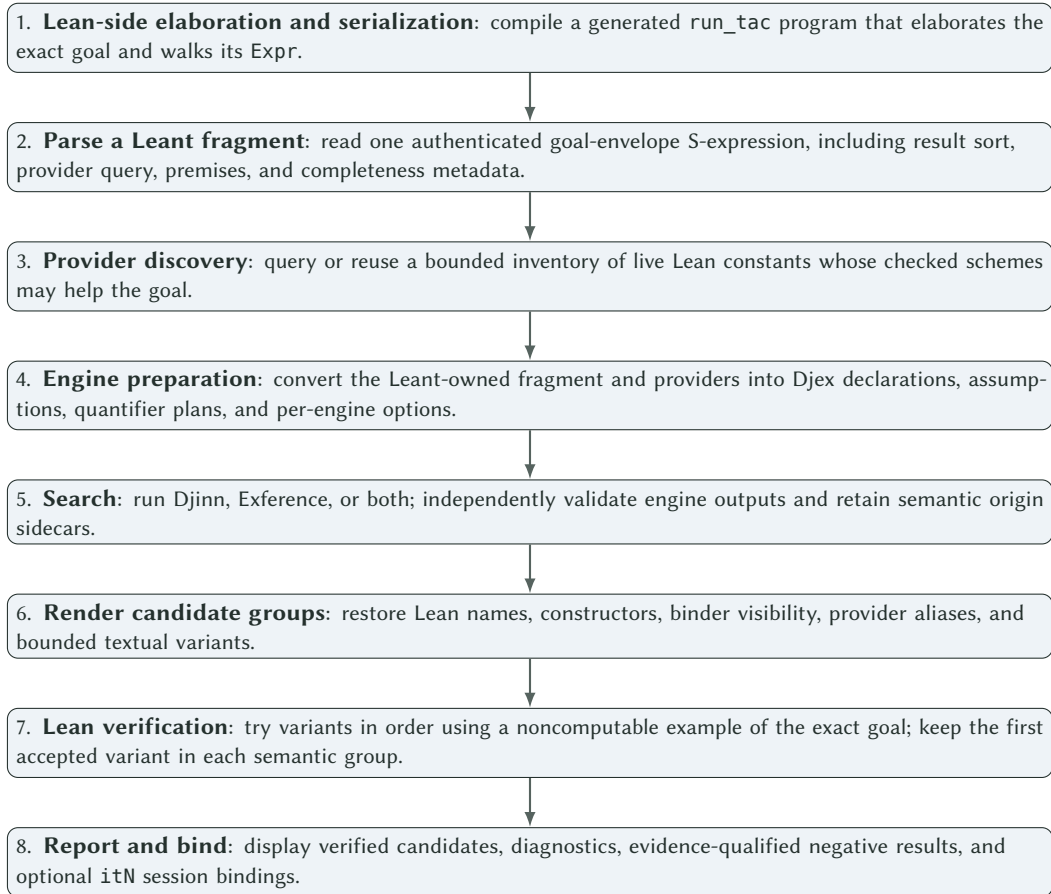
The Haskell state tracks, among other things:

- current Lean environment identifiers;
- replayable command history and session snapshots;
- synthesized `it`, `it1`, `it2`, ... bindings;
- engine selection, step/queue/budget settings, and result quota;
- provider inventories and cache keys tied to the Lean environment;
- library, classical, and optional behavioral settings;
- proof-mode state.

The persistent worker matters because a synthesis query is evaluated in the live environment created by previous commands and imports, not in a freshly guessed module.

17.2 The major stages

A representative `:synth GOAL` request follows this path:



17.3 Why translation happens inside Lean

The serializer is emitted as Lean source and executed in the live worker. It can therefore:

- elaborate the user’s exact text with Lean’s parser, macros, notation, imports, and options;
- instantiate metavariables and weak-head normalize under Lean’s definitions;
- inspect `Expr.forallE` dependency and binder information;
- resolve constant names and universe instantiations;
- inspect inductive declarations and constructor types;
- distinguish classes, propositions, data sorts, local variables, and nominal applications;
- use Lean’s definitional equality when comparing provider assignments.

Attempting these tasks by parsing pretty-printed goals in Haskell would immediately lose binding identity, namespace resolution, implicit arguments, reducible definitions, and universe information.

17.4 The serializer output is treated as an untrusted protocol

The generated Lean program emits a tagged S-expression inside an info message. User code can also emit messages, so Leant does not trust the first line with a familiar prefix. The parser requires exactly one well-formed goal envelope and rejects missing, duplicate, malformed, or trailing data.

This is a small but representative security principle: a semantic stage hands structured data to the next stage through a narrow checked boundary. Messages, strings, and debug output never become authority merely because they resemble internal protocol text.

17.5 Semantic origin survives textual operations

A candidate is not only a string. Leant records the exact source goal, search goal, declarations, providers, constructor/name maps, premise layout, engine preparation, search policy, and typed candidate or proof provenance needed to interpret it.

Rendering can produce several strings from one candidate. Textual deduplication can merge equal display forms. Verification can accept only one variant. Through those operations, the semantic origin sidecar must remain attached to the group that owns it. The internal documentation calls this a semantic-origin record; typed candidates additionally carry graph fingerprints and sealed inventories.

Authority does not transfer by string equality

Two identical Lean strings generated from different environments or candidate graphs are not automatically the same semantic artifact. A copied string has no right to inherit a completeness proof, behavioral contract, provider inventory, or solver receipt from another candidate merely because the characters match.

17.6 Backend verification usually dominates latency

Structural search can be fast, especially for small formulas. Candidate verification invokes Lean elaboration in the live environment for each variant and can dominate wall-clock time. Leant therefore groups variants, uses lazy quota-aware verification, caches stable provider inventories, and avoids counting failed variants against the accepted-result quota.

Source trail

The process protocol and lifetime management are in <https://github.com/VladimirReshetnikov/Leant/blob/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0/src/Leant/Backend.hs>. The interactive orchestration is in `src/Main.hs`. Translation and verification command generation are in `src/Leant/Synth/Fragment.hs`; internal provenance invariants are summarized in `docs/synth-internals.md`.

Chapter checkpoint

Leant is a persistent Haskell controller around an authoritative Lean worker. The goal is elaborated and classified inside Lean, searched in Djex, reconstructed in Haskell, and checked again inside Lean. Structured origins and exact environment identities survive every intermediate transformation.

The Leant Synthesis Fragment

18.1 A frontend-owned semantic tree

Leant does not translate Lean expressions directly into Djex types. It first constructs its own Frag tree. This keeps Lean-specific policy out of Djex and makes the exact/opaque/completeness decisions inspectable before an engine is chosen.

The principal fragment forms at the pinned revision are summarized below.

Fragment	Meaning	Typical Lean source
FArr	implication/function arrow	non-dependent $A \rightarrow B$
FProd	logical product	And, structural proof product
FLeanProd	Lean data product	Prod/PProd distinction retained for rendering
FSum	logical/data sum	Or, Sum, or PSum with constructor map
FTop	unit/truth	True, Unit, PUnit
FBot	empty/falsity	False, Empty, PEmpty
FAll	admitted sort quantifier	type/sort binder plus explicitness flag
FInst	instance binder wrapper	hidden class evidence retained as a semantic boundary
FExactContext	structured class context	exact provider schemes/assignments only
FVar	universal type variable	selected sort-bound local variable
FAtom	opaque atom with refutation-safety bit	dependent or unsupported closed subtree
FApp	nominal/variable type application	first-order type-valued application with retained arguments
FParamInd	parameterized nonrecursive inductive	complete sum-of-products family with parameters
FInd	nonparameterized nonrecursive inductive	complete constructor inventory
FParamRec	parameterized recursive family	bounded recursive description plus inventory completeness
FRec	recursive family	bounded recursive description plus inventory completeness
FDepth	bounded-depth sentinel	marks a traversal limit rather than logical structure

The exact constructor names matter less than the separation of concerns: arrows/products/sums are searchable logic; variables and applications preserve type structure; inductive records preserve declarations; atoms preserve identity; flags preserve whether absence of a proof can be trusted.

18.2 Recognized logical structure

The Lean-side traversal recognizes common families by exact constant identity, not by pretty-printed notation. It structurally exposes:

- non-dependent arrows;
- And, Prod, and PProd;
- Or, Sum, and PSum;
- Iff, lowered to the two implications;

- Not, lowered to implication into falsity;
- False/Empty/PEmpty;
- True/Unit/PUnit;
- selected sort quantifiers;
- qualifying inductive declarations.

The renderer later restores nominal distinctions and exact constructor names. The search engines can reason over a common product/sum structure without pretending the Lean families are definitionally identical.

18.3 Non-dependent versus dependent forall

For a Lean `forallE`, the serializer asks whether the body contains the bound variable.

Non-dependent value binder

Becomes an arrow: introduce an ordinary term and synthesize the codomain.

Sort-bound dependent binder

May become `FALL`: treat the bound object as a rigid type/sort variable and translate the body under it.

Instance-implicit binder

Becomes `FInst` or exact context metadata under narrowly admitted provider rules.

Other dependent value binder

Cannot become an ordinary arrow. The whole type or affected subtree is sealed as an opaque atom.

Thus $\forall \alpha : \text{Type}, \alpha \rightarrow \alpha$ can be represented as polymorphism, while $\forall n : \text{Nat}, P\ n$ is not analyzed as a first-order type scheme. It may still be carried opaquely.

18.4 Type applications retain arguments when they are type-level

Earlier versions of many synthesis adapters flatten applications into strings. Leant instead retains the application head and argument fragments when Lean establishes that the application is type-valued and its arguments fit the admitted first-order kind discipline.

For example, `List  $\alpha$` , `Except  $\varepsilon$   $\alpha$` , or a user-defined unary type constructor can preserve nominal family identity and parameter structure. This supports higher-kinded provider matching and prevents unrelated instantiations from collapsing into one atom.

A term-indexed application such as `Vector  $\alpha$   $n$` is different: n is data, not a type parameter. Unless the whole family is handled through a qualifying inductive representation, the application remains opaque. The search engine does not erase n and pretend all vector lengths coincide.

18.5 Opaque dependent cargo

Consider:

A dependent proposition moved as a whole

```
opaque P : Nat → Prop
opaque Q : Prop

-- Synthesis target:
(( $\forall n : \text{Nat}, P\ n$ )  $\wedge$  Q)  $\rightarrow$  Q  $\wedge$  ( $\forall n : \text{Nat}, P\ n$ )
```

Let X denote the exact opaque atom for $\forall n, P\ n$. The searchable formula is

$$(X \wedge Q) \rightarrow (Q \wedge X).$$

Djinn can synthesize the product swap. The renderer restores the original dependent formula as the type of the moved hypothesis; no term inspects n or proves any $P\ n$.

This technique is useful for arbitrarily complicated formulas when their role is purely structural. It is not dependent proof search. A target such as

Requires analysis, not transport

$(\forall n : \text{Nat}, P\ n) \rightarrow P\ 0$

requires instantiating the dependent function at 0. Whether Leant can handle a bounded form depends on its quantified-instantiation support and exact fragment plan; merely sealing the premise as one atom would make the needed application invisible.

18.6 Refutation-safety bits

An opaque atom can be safe for positive transport yet unsafe for negative reasoning. Leant attaches a Boolean safety classification to atoms and applications.

A formula built only from universally quantified abstract variables can sometimes be treated parametrically: its hidden structure cannot smuggle in a concrete known inhabitant. An opaque subtree mentioning concrete constants, fixed indices, hidden instances, incomplete declarations, or traversal limits may conceal exactly the proof that search failed to find. Such a subtree downgrades a negative outcome.

The flag is not Lean’s unsafe declaration marker. It means “safe to rely on this opacity when interpreting exhaustive failure.”

18.7 Depth and resource limits are represented, not forgotten

A bounded traversal that runs out of fuel emits a depth marker or completeness downgrade. It does not silently replace unseen structure with a trustworthy atom. This preserves the distinction between:

- a semantically opaque subtree intentionally treated as an atom;
- a subtree that was never fully inspected because a resource bound stopped the serializer.

Only the first can participate in carefully justified parametric negative reasoning, and even then only under its safety classification.

Source trail

The fragment datatype, generated Lean serializer, S-expression parser, provider metadata, and candidate-verification program are in <https://github.com/VladimirReshetnikov/Leant/blob/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0/src/Leant/Synth/Fragment.hs>. The module comments give the most concise current statement of the exact structural fragment and opaque-cargo policy.

Chapter checkpoint

Frag is Leant’s semantic firewall. It exposes admitted arrows, products, sums, polymorphism, applications, and inductives; seals unsupported dependency as alpha-aware atoms; and carries explicit completeness/safety metadata. It is richer than Djinn formulas but intentionally poorer than Lean expressions.

Inductives, Recursion, Providers, and Instances

19.1 Expanding an inductive as a sum of products

A nonrecursive datatype such as

A simple finite datatype

```
inductive Choice (A B : Type) where
| left  : A → Choice A B
| right : B → Choice A B
```

has the structural shape $A + B$. A one-constructor record with two independent fields has the shape $A \times B$. Leant may inspect the live inductive declaration and emit constructor-tagged products and sums when all of the following are established:

- the family is nonindexed for the admitted use;
- constructor results target the same family parameters;
- relevant fields are explicit and non-dependent;
- the complete constructor inventory is available;
- kind and universe shape fit the fragment policy;
- no hidden field or unsupported occurrence invalidates reconstruction.

Constructor names and arities are retained so a Djinn proof term can be lowered back to the correct Lean injections and case branches.

19.2 Why indexed and dependent-field inductives remain opaque

For `Eq a b`, the only constructor changes what can be known about the index b . For `Vector α n`, constructors return different indices. For `Exists`, a later proof field depends on the witness. Flattening these to ordinary sums/products would discard the equalities and dependencies that make their eliminators type-correct.

Leant therefore keeps such families opaque unless a dedicated representation accounts for the dependency. The policy favors an incomplete search over an unsound translation.

19.3 Recursive types and one-layer structure

A recursive declaration such as `List α` is structurally

$$1 + (\alpha \times \text{List } \alpha).$$

This equation suggests constructor introduction and case analysis, but unrestricted proof search can unfold it forever. Leant and Djex therefore retain a bounded recursive description with explicit recursive occurrences and inventory completeness.

Current behavior distinguishes:

- bounded positive recursive introduction, which can construct finite layers;
- recursive elimination, supported only under restricted Exference policies;
- general recursion or recursor invention, which is not part of ordinary structural search.

A recursive family whose inventory is partial may still produce a candidate that Lean verifies. It cannot support a complete “uninhabited” conclusion.

19.4 Providers turn the environment into premises

A synthesis problem is often impossible from local arguments alone but easy using an imported function. Leant discovers selected global constants as *providers*. A provider contributes a checked type scheme and an exact Lean global name; the engine may use it like an additional premise.

For example, a goal requiring `List.reverse` could be solved only if that constant is in the admitted provider inventory. Provider discovery is bounded and query-directed so the search environment does not explode with every declaration in every import.

The provider query records nominal roots and a possible result head. The Lean-side scanner uses the actual environment, elaborated types, kind arities, binder visibility, and instance constraints to identify candidates. Haskell caches inventories only under environment identities that make reuse valid.

19.5 Curated library mode

The `synth-library` setting exposes a curated collection of useful library functions and recursors. This is a pragmatic way to synthesize nontrivial behavior without teaching the core engine to invent recursion.

The distinction is important:

- using a known provider is ordinary application synthesis;
- inventing the provider’s recursive implementation is program synthesis at a much richer level.

A term using an admitted provider is still verified against the exact Lean goal. Negative conclusions are relative to the provider inventory and its completeness policy, not to every constant that might exist in Lean.

19.6 Provider schemes and exact instantiations

A polymorphic provider may have sort binders and class constraints. Leant’s provider protocol records:

- the exact global name;
- a fragment for its source scheme;
- binder metadata aligned with visible quantifiers;
- exact or legacy bounded instantiation evidence for erased constraints;
- nominal higher-kinded arguments with admitted kind arities;
- structured class contexts only at the provider-owned boundary.

The generated Lean code asks the live instance resolver for candidate instance heads, explores each under restored metavariable state, closes all related constraints, and keeps only complete ground-kinded assignments. Partial assignments do not enter the search pool.

This machinery allows a provider whose applicability depends on type-class evidence to be instantiated more faithfully than a flat collection of guessed type arguments.

19.7 Why instance search must be bounded and fail closed

Type-class resolution can recurse through instance premises, branch among heads, and instantiate type variables. An exhaustive global characterization may be impractical or impossible under open-world imports. Leant therefore uses explicit caps on inspected heads, assignments, argument counts, and kind arities.

Those bounds are useful for positive synthesis: any resulting term is checked. They are not silently upgraded into a proof that no other instance assignment exists. Exhaustion or omission is recorded in the completeness ledger.

19.8 Premise staging

Leant may run multiple search stages:

- local goal structure and hypotheses;
- discovered providers;
- curated library providers;
- optional classical premises;
- engine-specific recursive support.

A conclusive negative claim must account for every stage that is part of the configured search contract. Failure in an early stage is not final if a later provider stage is still permitted.

Open-world versus closed-world reading

Lean's environment is open and extensible: a future import can add a function of the desired type. Leant's refutation can at most concern the closed structural calculus and exact provider inventory sealed for the current query. It is not a metaphysical claim that no Lean declaration could ever inhabit the type.

Chapter checkpoint

Simple complete inductives can become sums of products; indexed or dependent families remain opaque. Recursive structure is bounded. Providers make selected live declarations available as premises, with exact names and checked schemes. Type-class assignments and discovery bounds help positive search but weaken negative evidence unless exhaustiveness is established.

Djinn and Exference: Two Different Search Contracts

20.1 Why Leant has two engines

No single search strategy is best for every goal. Djinn offers a logical decision procedure for a disciplined fragment. Exference offers a richer, ranked, resource-bounded expression search. Leant can run either engine or merge both result streams, but it must not merge their evidence meanings.

20.2 Djinn: proof search in LJ_T

Djinn uses Roy Dyckhoff’s contraction-free LJ_T calculus for intuitionistic propositional logic. The calculus is designed to avoid unrestricted contraction and to make proof search terminate without repeatedly duplicating assumptions.

At a high level, Djinn:

1. converts a checked source type and declaration environment into an LJ_T formula and named assumptions;
2. applies right rules to decompose the goal;
3. applies focused left rules to exploit assumptions;
4. produces a normalized proof term with free variables corresponding to premises;
5. independently checks the proof term against the formula environment;
6. lowers the proof through constructor and provider mappings into a typed candidate representation.

The default unbudgeted depth-first mode is complete for the supported formula calculus. An optional choice-point budget changes the contract: an empty result with budget exhaustion means only “not found within the explored prefix.”

Djinn can retain alternative proofs and can interleave branches under another strategy, but search strategy is distinct from logical completeness. The code records whether a bound stopped exploration.

20.3 What Djinn can decide

For an exact formula made of atoms, implication, finite conjunction/disjunction, and nominal emptiness, unbudgeted LJ_T search can decide intuitionistic provability. By Curry–Howard, it can produce a structural inhabitant or establish non-provability in that formula calculus.

This is not yet a Lean non-inhabitation result. Leant must additionally establish that:

- its Lean-to-fragment translation was exact for negative purposes;
- fragment-to-Djinn lowering preserved the relevant semantics;
- all premises and constructor declarations were complete;
- no bounded quantifier/provider/recursive stage remained;
- search was genuinely unbounded and exhausted.

Only then may Djinn’s non-provability support a proof-backed uninhabited report.

20.4 Exference: best-first typed expression search

Exference explores typed expression states using a priority queue and heuristic scores. It combines function bindings, deconstructors, unification, constraints, rigid instantiation plans, expression checking, and search-control policies.

Typical controls include:

- maximum processed search nodes;
- maximum queue size;
- optional heuristic-depth cap;
- whether unused bindings are allowed;
- whether unresolved constraints may be deferred;
- whether to pattern-match multi-constructor datatypes;
- heuristic penalties and ranking configuration.

Its completion status distinguishes full exhaustion from step limits, queue/depth pruning, and identifier-space exhaustion. This is good operational hygiene, but even full exhaustion is conclusive only for the sealed Exference calculus and finite environment. Leant currently treats Exference as a positive heuristic engine, not a refutation authority.

20.5 Typed candidate graphs

Modern Djex retains more than rendered source. An Exference candidate can carry a typed term graph, source type-variable hints, exact inventory identity, and fingerprints. Independent checkers validate type, scope, syntax, constraints, and source projection before the candidate enters canonical result envelopes.

Leant preserves this sidecar through grouping and verification. It enables later semantic layers, such as Length interpretation, to analyze the exact candidate structure rather than reparsing a Lean string.

20.6 Result merging

When Leant runs both engines, candidate streams may contain extensionally or textually duplicate answers. Merging must preserve:

- engine identity and search policy;
- semantic candidate identity;
- rendering variants;
- exact typed origin where available;
- notes about recursive/provider/classical routes;
- verification status.

A duplicate string from Djinn and Exference can be displayed once while still retaining distinct provenance internally. In particular, Exference must not inherit Djinn’s completeness evidence merely by producing the same text.

Property	Djinn	Exference
Core strategy	contraction-free logical proof search	heuristic best-first expression search
Main strength	small exact structural formulas	broader/ranked practical candidates
Unbounded decision procedure	yes, for admitted LJT formula fragment	no general Leant refutation contract
Resource controls	alternatives, strategy, optional choice budget	steps, queue, depth, constraints, heuristics
Proof/candidate validation	independent LJT proof checker and typed lowering	independent typed expression/candidate checks
Negative authority in Leant	possible only with complete translation ledger	none

Source trail

Djinn's search contract is documented in <https://github.com/VladimirReshetnikov/Djex/blob/0cf82f2aa22debb1de2c7aaa13a2abbe190550d8/djinn/src-core/Djinn/Internal/LJT.hs>. Exference's validated input, priority-queue search, completion states, and projections are in `exference/src-core/Language/Haskell/Exference/Core/Internal/Exference.hs`. Leant's engine boundary is `src/Leant/Synth/Engine.hs`.

Chapter checkpoint

Djinn and Exference share source infrastructure but promise different things. Djinn can decide an exact propositional fragment when unbounded. Exference is a ranked, bounded positive search. Leant may merge candidates, never evidence authorities.

Rendering Candidates Back into Lean

21.1 A proof term is not yet Lean source

Djinn returns a proof term over symbols, tuple/sum operations, and premise variables. Exference returns an expression graph in its own syntax. Neither is directly accepted by Lean. Rendering must reconstruct:

- lambda binders with legal, noncapturing names;
- applications and precedence;
- exact Lean global names for providers and constructors;
- product construction and splitting;
- injections and case expressions;
- impossible elimination from empty types;
- explicit versus implicit type binders;
- missing type arguments and instance arguments;
- nominal type-family distinctions erased by logical lowering.

21.2 Universe-agnostic constructor syntax

Whenever possible, the renderer uses syntax whose meaning is supplied by the expected goal:

- `{a, b}` for one-constructor product-like targets;
- dot constructors for sum injections when the expected family identifies them;
- `match` for case analysis;
- `nomatch` or empty elimination for impossible cases.

This avoids hard-coding universe-specific constructors and lets Lean recover exact parameters.

21.3 Binder reconstruction

Djinn's formula quantification does not remember every Lean surface binder. Leant records a visibility vector and walks the candidate term against it.

An explicit type binder may require an actual lambda in rendered source:

Explicit type argument

```
fun (α : Type) => ...
```

An implicit type binder may remain omitted because Lean can infer it. A provider application may require an explicit placeholder:

Explicit slot restored by a placeholder

```
someProvider _ x
```

If a quantified argument can be instantiated in more than one plausible place, Leant generates a bounded family of textual variants.

21.4 One semantic candidate, several strings

Suppose the engine has proved the structural candidate “apply provider p to value x .” Depending on Lean’s original binder sequence, viable renderings may include:

Illustrative variant group

```
p x
p _ x
@p _ x
```

These strings are not three independently discovered programs. They are reconstruction attempts for one semantic candidate. The verification loop tries them in deterministic order and accepts the first that elaborates.

Grouping matters for quotas: failed variants do not consume a requested candidate slot, and multiple accepted spellings of the same semantic term do not flood the output.

21.5 Name restoration and hygiene

Search engines allocate their own binder symbols and may alias provider names. The renderer:

- restores exact global Lean names from checked maps;
- allocates fresh local names avoiding collisions with globals, atoms, and other binders;
- respects shadowing and alpha-equivalence;
- uses constructor descriptors validated against declarations;
- avoids treating opaque display text as an identifier.

The result must be syntactically valid even before type checking, but syntax validity never grants semantic authority.

21.6 Rendering failure is ordinary candidate failure

A search result can be valid in the smaller calculus yet lack a supported Lean rendering. For example, a legacy proof-term node may have no current lowering, or a quantified instantiation may be irrecoverably ambiguous. Leant drops that rendering route and records diagnostics rather than inventing unchecked syntax.

The renderer is intentionally untrusted

A renderer bug can produce malformed or ill-typed Lean source. The verification boundary converts that bug into a rejected candidate. The design does not require the renderer to be part of the logical trusted base.

Source trail

Candidate lowering, binder visibility, variant enumeration, name restoration, and universe-agnostic Lean syntax are implemented in <https://github.com/VladimirReshetnikov/Leant/blob/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0/src/Leant/Synth/Render.hs>.

Chapter checkpoint

Rendering is a reconstruction problem, not pretty-printing. Leant may derive several Lean strings from one semantic candidate, preserves exact names and binder hygiene, and relies on expected-type elaboration. Unsupported or wrong renderings are simply rejected.

Verification and the Positive Soundness Boundary

22.1 The exact verification program

For a goal string G and rendered candidate t , Leant constructs:

Candidate verification

```
set_option autoImplicit true in
noncomputable example : (G) := (t)
```

This command is compiled in the current Lean environment. It requires:

- the goal to elaborate exactly as a type;
- the candidate to elaborate against that expected type;
- all names, implicit arguments, instances, coercions, and universes to resolve;
- the elaborated term to contain no unsolved metavariables;
- the kernel to accept the resulting declaration under safety rules.

A successful callback is wrapped as `Verified` inside Leant's verification module. That wrapper means only that this callback accepted this variant; it is not a behavioral proof, solver certificate, or completeness witness.

22.2 Lazy quota-aware checking

Candidates arrive in semantic groups. Verification proceeds lazily:

1. take the next group;
2. try its variants in deterministic order;
3. on the first accepted variant, emit one verified candidate and consume one quota slot;
4. if every variant fails, continue without consuming the quota;
5. stop once the requested number of accepted groups is reached or the stream ends.

This avoids eagerly compiling a potentially large candidate stream and gives users the requested number of usable answers rather than the requested number of attempts.

22.3 What positive verification guarantees

If the Lean worker accepts the command, then relative to the live environment:

$$\vdash t : G.$$

This guarantee survives errors in:

- fragment classification;
- Djex search;
- proof-term lowering;
- provider matching;
- name restoration;
- variant selection.

Those errors may cause missed candidates, wasted work, or surprising terms, but an ill-typed result should not be displayed as verified.

22.4 What positive verification does not guarantee

It does not establish that:

- the candidate has the user’s intended behavior;
- the candidate is efficient, terminating under unsafe execution paths, or free of undesirable axioms;
- the candidate is unique or minimal;
- the search was complete;
- the candidate satisfies an optional semantic contract not encoded in its Lean type;
- a textual copy in another environment will still elaborate.

A type such as `List  $\alpha$  → List  $\alpha$` admits many verified inhabitants. Verification proves type correctness, not intent.

22.5 Binding accepted candidates into the session

Leant can retain accepted results as `it` or numbered `itN` declarations. The binding is created through the Lean backend, giving later commands an exact environment object rather than a Haskell-side fiction. Replay and snapshot machinery preserve the command history needed to reconstruct session state.

Never reverse the trust arrow

The correct reasoning is “Lean accepted the candidate, therefore it inhabits the goal.” It is not “Djex produced the candidate, therefore Lean ought to accept it.” Search is advisory; verification is authoritative.

Source trail

The group-level lazy verifier and its narrow `Verified` wrapper are in <https://github.com/VladimirReshetnikov/Leant/blob/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0/src/Leant/Synth/Verification.hs>. The exact Lean command is generated in `src/Leant/Synth/Fragment.hs`; session binding is orchestrated by `src/Main.hs`.

Chapter checkpoint

Every displayed candidate crosses the original Lean elaboration and kernel boundary. Verification is lazy and group-aware. It proves exact type inhabitation in the live environment, while deliberately making no claim about behavior, optimality, or search completeness.

Negative Evidence and the Completeness Ledger

23.1 Why “no term found” has several meanings

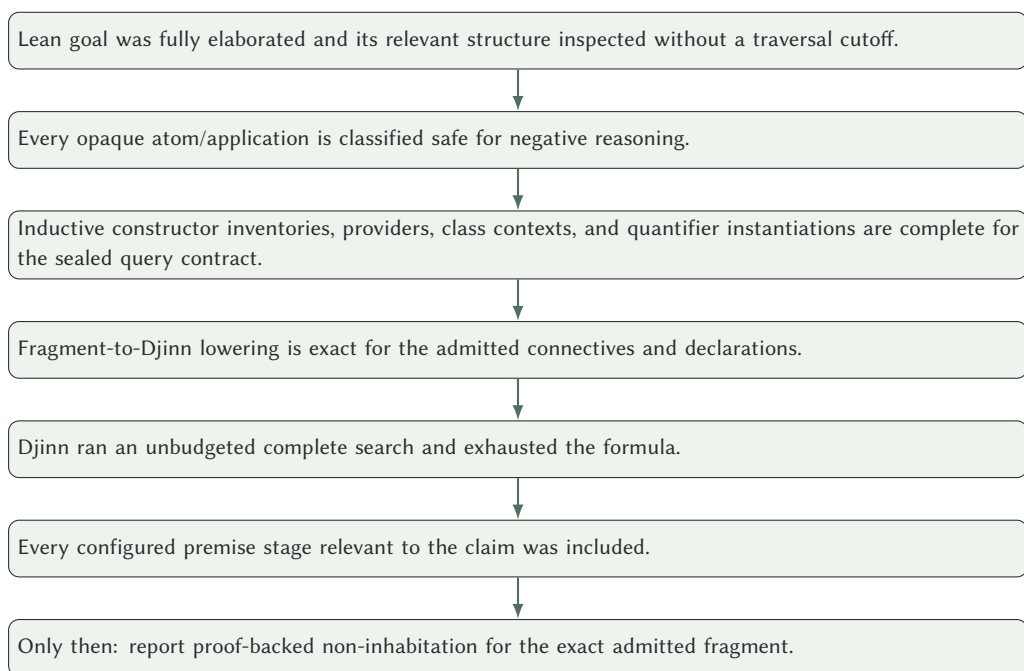
An empty candidate list can result from very different events:

- the exact structural formula is intuitionistically unprovable;
- the search budget expired;
- Exference pruned its queue or reached a depth cap;
- provider discovery omitted a useful constant;
- a quantified instantiation plan was bounded;
- recursive structure was only partially unfolded;
- hidden instance evidence was not enumerated;
- a valid engine candidate had no supported rendering;
- every rendered variant failed Lean elaboration;
- the goal contains dependent structure represented opaquely.

Only the first event, under an exact translation and complete environment contract, supports “uninhabited.” The others support weaker statements such as “no verified term was found under current bounds.”

23.2 The chain required for a strong negative result

A proof-backed negative report requires all links below:



A single failed link downgrades the wording.

23.3 Exact, safe opacity versus unsafe opacity

Opacity is not automatically fatal. If a universally quantified abstract formula is sealed as an atom and the structural calculus treats it parametrically, exhaustive failure may still be meaningful. But an opaque concrete type can hide a constructor or theorem. For example, treating `Nat` as an arbitrary atom and searching for `Nat` from no premises would return no proof, although `0` is an inhabitant.

Leant's safety bit and completeness facts distinguish these cases. Concrete constants and fixed applications generally prevent a structural absence from becoming a Lean-level refutation unless their constructor inventory is represented exactly.

23.4 Quantifier plans

Higher-rank and impredicative synthesis may require instantiating polymorphic premises. A bounded finite plan can discover useful candidates, but failure after finitely many guessed instantiations is not a proof that no instantiation works.

The completeness ledger records whether the current quantifier fragment and plan are among the exact complete cases. Ambiguous trailing instantiations that are merely rendered as variants also cannot support negative authority.

23.5 Provider-relative completeness

A closed formula may be uninhabited from local assumptions but inhabited using a global provider. Leant's query contract therefore includes the provider stages and inventories actually searched.

Even a complete provider scan over one live environment would establish only environment-relative absence. Leant uses bounded query-directed discovery, so most provider-enabled failures remain explicitly inconclusive.

23.6 Recursive and instance-related downgrades

A recursive type with bounded unfolding may hide arbitrarily deep constructor terms. A hidden instance dictionary may carry methods or data. A partial constructor inventory may omit a branch. Each is acceptable for positive search because Lean checks candidates; each blocks strong negative interpretation.

23.7 Djinn versus Exference wording

Djinn can produce an internal evidence value corresponding to proved uninhabitability only when its formula search is complete. Leant still gates that value through fragment safety.

Exference never grants Leant a refutation. Even if its queue reports natural exhaustion, the engine is used as a heuristic candidate generator under a finite environment and option set. Leant reports absence, limits, and pruning diagnostics without calling the Lean type uninhabited.

A useful three-valued mental model

For a synthesis query, think in three broad outcomes:

1. **verified inhabitant**: a concrete Lean term checked;
2. **proved uninhabited in the exact admitted contract**: a rare, evidence-backed negative;
3. **unknown under current translation/search bounds**: the normal meaning of exhaustive-looking failure outside that contract.

23.8 Failure transparency

Good diagnostics report why authority was lost: unsafe opaque atoms, hidden instances, recursive partiality, bounded search, provider limits, rendering rejection, or Lean elaboration errors. Leant's debug mode can expose the fragment, providers, rendering variants, engine notes, and metrics needed to understand a result.

Source trail

The refutation gate is owned by <https://github.com/VladimirReshetnikov/Leant/blob/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0/src/Leant/Synth/Engine.hs>; fragment safety originates in `src/Leant/Synth/Fragment.hs`. Djinn's budget/exhaustion distinction is in `Djinn/Internal/LJT.hs`; Exference's completion states distinguish natural exhaustion, limits, pruning, and identifier exhaustion.

Chapter checkpoint

A negative result is a theorem about a search contract, not the absence of output. Leant needs exact translation, safe opacity, complete inventories and premise stages, and unbounded Djinn exhaustion. Otherwise the correct result is unknown or not found within bounds.

The Interactive Layer: `:synth`, `:prove`, Sessions, and Debugging

24.1 Expression-oriented interaction

Leant is designed for rapid exploration. Users can submit Lean declarations and expressions, inspect types and values, synthesize terms, enter proof workflows, and refer to previous results through `it`-style bindings.

The Haskell frontend remembers the Lean environment identifier after each accepted command. A failed speculative command need not destroy the prior state. History and snapshots support replay into a fresh backend process when necessary.

24.2 `:synth`

The synthesis command asks for whole terms inhabiting a goal. Relevant controls include:

- engine selection: Djinn, Exference, or both;
- number of accepted candidates;
- search steps, queue/depth limits, and alternative policy;
- provider and library settings;
- classical fallback;
- debug output;
- optional Length assessment.

The command is best suited to structurally informative types: adapters, combinators, product/sum rearrangements, polymorphic plumbing, constructor assembly, and applications of known providers.

24.3 `:prove`

Proof mode works with goals in an interactive Lean proof state. It can use synthesis to propose a term for a current goal, but it also benefits from ordinary Lean tactics for rewriting, induction, simplification, arithmetic, and dependent case analysis.

This division reflects the theory:

- `:synth` searches for an inhabitant of a comparatively self-contained target;
- `:prove` can transform a difficult dependent goal into simpler structural subgoals before synthesis is invoked.

A vector theorem may first require induction and index rewriting; a residual branch might then reduce to a product/function wiring problem that Djinn handles well.

24.4 Provider caches and environment identity

Provider discovery can be expensive. Leant caches inventories under keys derived from the live environment and query information. A cache hit is valid only if the exact environment assumptions remain stable. Replaying a session or changing imports can invalidate provider worlds.

The cache stores checked semantic records, not merely names. Reusing a string list across environments could associate a name with a different declaration or miss new instances.

24.5 Debugging a missing candidate

When an expected term does not appear, inspect the pipeline in order:

1. Did the raw goal elaborate as intended?
2. Which subtrees became arrows/products/sums, applications, inductives, or opaque atoms?
3. Was a depth or kind limit reached?
4. Were the needed providers discovered, and under what schemes?
5. Which engine ran, with what bounds?
6. Did the engine produce a semantic candidate?
7. Which Lean variants were rendered?
8. Why did Lean reject them?

This avoids blaming “search” for a failure caused by provider discovery or rendering, and avoids blaming “Lean” for a candidate never generated.

24.6 Reading source ownership

A useful source map is:

Concern	Primary Leant source
interactive state and command orchestration	src/Main.hs
Lean process protocol and timeouts	src/Leant/Backend.hs
Lean-side goal/provider serialization	src/Leant/Synth/Fragment.hs
fragment-to-Djex preparation and search	src/Leant/Synth/Engine.hs
Djex-term-to-Lean reconstruction	src/Leant/Synth/Render.hs
lazy group verification	src/Leant/Synth/Verification.hs
semantic provenance invariants	docs/synth-internals.md
user-facing commands and examples	README.md, docs/Leant.tex

For Djex, begin with README.md, then shared `synthesis/src/Language/Haskell/Synthesis`, Djinn’s `Internal/LJT*.hs`, and Exference’s `Core/Internal/Exference.hs`.

Chapter checkpoint

The REPL layer preserves a live Lean environment and exposes synthesis as one tool among ordinary declarations and proof operations. Debugging is easiest when the pipeline stages are inspected separately. Source ownership is intentionally narrow enough that each semantic boundary has a clear home.

Part V

Worked Traces and Boundaries

Worked Structural Synthesis Traces

This chapter follows representative goals through the conceptual pipeline. The exact debug spelling may evolve, but the semantic stages are fixed by the pinned source.

25.1 Composition

Consider:

Leant query

```
:synth ((A → B → C) → (A → B) → A → C)
```

Lean elaboration

With auto-implicit variables, Lean elaborates A , B , and C as implicit sort variables and the goal as nested Π -types. The value binders are non-dependent arrows.

Fragment

The structural fragment is

$$(A \rightarrow B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow A \rightarrow C,$$

with A, B, C rigid variables. No opaque atoms, inductives, providers, instances, or bounds are needed.

Djinn formula and proof search

The formula is the same propositional implication shape. Right rules introduce f , g , and x . The atomic goal is C .

The context contains:

$$f : A \rightarrow B \rightarrow C, \quad g : A \rightarrow B, \quad x : A.$$

To use f , search must provide A and B . It uses x for A and applies g to x for B . The proof term is equivalent to

$$\lambda f. \lambda g. \lambda x. f x (g x).$$

Rendering and verification

The renderer chooses fresh Lean binder names and emits:

Candidate

```
fun f g x => f x (g x)
```

Lean checks it against the exact original goal. Because the fragment was exact and Djinn’s unbudgeted search is complete, failure could also have had a strong logical interpretation; in this case a verified positive answer ends the story.

25.2 Swapping a product

For data:

Product swap

```
:synth (A × B → B × A)
```

or for propositions:

Conjunction swap

```
:synth (P ∧ Q → Q ∧ P)
```

both lower to a product-swap shape. The exact nominal families and result sort are retained outside the formula calculus. Djinn’s proof term splits the input pair and constructs a pair in reverse order.

The renderer can use expected-type syntax:

Universe-agnostic product syntax

```
fun p => ⟨p.2, p.1⟩
```

Lean resolves the appropriate constructor and projections from the expected type. Search shares a logical shape; verification restores the actual Lean family.

25.3 Sum reassociation

Consider:

A sum transformation

```
:synth (A ⊕ (B ⊕ C) → (A ⊕ B) ⊕ C)
```

The formula becomes a constructor-tagged disjunction. A proof must case-analyze the input:

Representative candidate

```
fun s =>
  match s with
  | .inl a => .inl (.inl a)
  | .inr (.inl b) => .inl (.inr b)
  | .inr (.inr c) => .inr c
```

The exact syntax may use qualified constructors depending on context, but the semantic proof contains three branches and tagged injections. Constructor descriptors preserved during lowering prevent the branches of unrelated sum-like inductives from being confused.

25.4 A local continuation pattern

The type

Continuation-shaped structural goal

$$((A \rightarrow B) \rightarrow A) \rightarrow (A \rightarrow B) \rightarrow B$$

is not intuitionistically inhabited in general: from $k : (A \rightarrow B) \rightarrow A$ and $f : A \rightarrow B$, one can derive $k f : A$ and then $f(k f) : B$. In fact it *is* inhabited, and Djinn can discover:

Candidate

```
fun k f => f (k f)
```

The example illustrates why visual intuition can be unreliable. Systematic proof search explores how assumptions' result atoms can feed one another.

Chapter checkpoint

For exact arrows, products, and sums, the pipeline closely follows Curry–Howard introduction and elimination rules. Lean-specific nominal details ride alongside the formula and return during rendering. Verification confirms that the reconstructed syntax matches the original family and binder structure.

A Complete Negative Trace

26.1 The target

Consider the polymorphic type:

Leant query

```
:synth (∀ A B : Type, A → B)
```

No assumptions, axioms, bottom values, unsafe operations, or providers are present in the closed goal.

26.2 Rigid quantification

Lean elaborates A and B as explicit type binders. Leant admits them as rigid universally quantified type variables. Rigid means that search cannot unify A with B merely to make the arrow look like identity.

The body is $A \rightarrow B$. Right introduction gives a hypothetical $x : A$ and leaves atomic goal B .

26.3 Exhaustive LJT failure

The context contains only $x : A$. There is:

- no hypothesis of type B ;
- no implication ending in B ;
- no product or sum whose elimination yields B ;
- no constructor inventory or provider for arbitrary rigid B ;
- no falsity from which to eliminate.

The LJT search space closes without a proof.

26.4 Why this failure can be conclusive

For this goal, the relevant ledger can be exact:

- only sort quantification and one ordinary arrow occur;
- the variables are abstract and rigid;
- no concrete opaque atom conceals an inhabitant;
- no indexed or recursive inductive is involved;
- no instance binder or provider stage is needed;
- Djinn can run unbudgeted.

Leant can therefore report proof-backed uninhabitation for the admitted constructive fragment, rather than merely “no term found.”

26.5 What would change the conclusion

Adding any of the following changes the problem:

Extra premises that make the target inhabitable

```
-- A direct provider:
(∀ A B : Type, (A → B) → A → B)

-- Falsity in the context:
False → (∀ A B : Type, A → B)

-- Classical choice plus a nonempty assumption for B:
∀ A B : Type, Nonempty B → A → B
```

The first returns the supplied function. The second eliminates falsity. The third may choose a B from nonemptiness under suitable classical principles. Refutation is always relative to the exact context.

Do not confuse parametric impossibility with runtime tricks

A partial language could implement $A \rightarrow B$ by looping forever or throwing an unchecked exception. Lean’s safe logical fragment requires total terms. Unsafe or partial executable definitions do not become proofs of arbitrary propositions.

Chapter checkpoint

The negative result works because the type translates exactly, universal variables stay rigid, no hidden structure or premise exists, and complete LJT search exhausts. This is the model case for Leant’s strongest negative wording.

Dependent Cargo: What Works Without Dependent Search

27.1 Moving an opaque dependent formula

Declare:

Dependent propositions

```
opaque P : Nat → Prop
opaque Q : Prop
```

and ask for:

Structural use of dependency

```
:synth (((∀ n : Nat, P n) ∧ Q) →
        (Q ∧ (∀ n : Nat, P n)))
```

Let

$$X = \forall n : \text{Nat}, P(n).$$

Leant seals the entire X as one alpha-aware atom. The formula seen by Djinn is $(X \wedge Q) \rightarrow (Q \wedge X)$. A product-swap proof is sufficient.

A representative Lean term is:

Candidate

```
fun h => ⟨h.2, h.1⟩
```

The term never inspects the proof of X . It merely returns the same proof object in another field. Lean's expected type confirms that $h.1$ has the exact dependent proposition.

27.2 Instantiating the dependent premise does not work by mere opacity

Now ask for:

Requires dependent application

```
(∀ n : Nat, P n) → P 0
```

The obvious term is:

Human-written term

```
fun h => h 0
```

But if the premise $\forall n, P\ n$ is sealed as one atom X and the goal $P\ 0$ as another atom Y , the propositional formula is $X \rightarrow Y$. No relationship between X and Y is visible. Moving opaque cargo is sound; opening it to perform dependent application requires a richer rule.

Leant has bounded support for selected quantified type instantiations, especially Haskell-like sort polymorphism. A value-indexed dependent forall over Nat is a different feature and should not be inferred from that support.

27.3 Transport is likewise invisible

Consider:

Transport target

```

∀ (α : Type) (n m : Nat),
  n = m → Vector α n → Vector α m

```

A human can write:

Human-written transport

```

fun α n m h v => h ▸ v

```

The essential step eliminates equality into a type family. A propositional search that treats $n = m$, $\text{Vector } \alpha\ n$, and $\text{Vector } \alpha\ m$ as unrelated atoms cannot synthesize it. Erasing the indices would be unsound; preserving them opaquely is safe but incomplete.

27.4 Use proof mode to expose structural subgoals

In a larger theorem, ordinary Lean tactics can perform induction or rewriting first. After substituting m for n , the remaining vector term may have exactly the expected type by definitional equality. If the residual goal is a function/product/sum wiring problem, synthesis can then help.

This is the intended collaboration:



A precise capability statement

Leant can often *transport values whose types are dependent* through surrounding structural logic. It generally cannot *reason inside those dependent types* by inventing indices, motives, equality proofs, or transports.

Chapter checkpoint

Opaque cargo supports rearrangement, storage, and application through already exposed premises. It does not reveal dependent elimination rules. Value-indexed foralls, equality transport, and indexed pattern matching remain jobs for Lean tactics or a future richer synthesis calculus.

Polymorphism, Rank-N Use, and Rendering Variants

28.1 A polymorphic argument

Consider:

Rank-N target

```
(( $\forall \alpha : \text{Type}, \alpha \rightarrow \alpha$ )  $\rightarrow$  Nat)
```

The argument f is itself polymorphic. One inhabitant is:

Candidate

```
fun f => f Nat 0
```

The type variable α is nested inside the domain of the outer function, so this is higher-rank use. Search must instantiate f at Nat and then provide 0 .

28.2 How the type crosses the boundary

Leant classifies the outer binder as a value arrow. The domain retains an explicit `ForallType` over a rigid type variable, with body $\alpha \rightarrow \alpha$. Djex’s higher-rank machinery and rigid instantiation plan may expose an application of f at an admitted ground type.

The natural-number result requires a constructor or provider. With the recursive `Nat` declaration represented sufficiently for positive introduction, zero is available. Positive success is safe even if recursive completeness is bounded, because Lean verifies the concrete term.

28.3 Why a renderer may need variants

The original binder $\forall \alpha : \text{Type}, \dots$ is explicit. A source scheme written with braces would be implicit:

Implicit polymorphic argument

```
( $\{\alpha : \text{Type}\} \rightarrow \alpha \rightarrow \alpha$ )  $\rightarrow$  Nat
```

Depending on the exact source binder, a candidate application might need `f Nat 0`, `f 0`, or `f ( $\alpha := \text{Nat}$ ) 0`. The engine’s abstract application does not by itself encode every surface choice. Leant’s side metadata and bounded variant enumerator reconstruct candidates and let Lean choose the valid one.

28.4 Ambiguity weakens negative evidence

If positive search finds a semantic application but every reconstruction variant fails, no verified candidate is shown. Conversely, a bounded instantiation plan that finds no semantic application does not prove none exists. Higher-rank positive support and higher-rank complete refutation are different capabilities.

Chapter checkpoint

Djex's source language can represent selected higher-rank type schemes even though it is not dependently typed. Leant records binder visibility and may enumerate application spellings. Positive candidates are verified; bounded instantiation and reconstruction prevent broad negative conclusions.

Provider-Guided and Library-Guided Synthesis

29.1 A global conversion function

Imagine a live Lean environment containing:

Environment declarations

```
opaque A : Type
opaque B : Type
opaque convert : A → B
```

A query for $A \rightarrow B$ has no closed structural inhabitant if A and B are unrelated atoms. If provider discovery admits `convert`, the search environment gains a premise of exactly that type. Djinn or Exference can produce:

Provider-backed candidate

```
fun x => convert x
```

The candidate's origin records that `convert` was a live provider, not a local lambda binder. The renderer restores its exact global name; Lean resolves the declaration in the current environment and checks the application.

29.2 Polymorphic providers

Suppose:

A polymorphic provider

```
opaque mapBox {α β : Type} : (α → β) → Box α → Box β
```

For a goal $(A \rightarrow B) \rightarrow \text{Box } A \rightarrow \text{Box } B$, provider matching must instantiate $\alpha := A$ and $\beta := B$. The exact type-constructor applications `Box A` and `Box B` must retain both nominal head and arguments; flattening them to unrelated text or a single `Box` atom would break unification.

A renderer may emit:

Provider application

```
fun f x => mapBox f x
```

and let Lean infer the implicit type arguments.

29.3 Class-constrained providers

A provider might require $[C \ \alpha]$. Lean's provider scanner can ask the live instance resolver for complete assignments and preserve structured exact context metadata. If a valid assignment exists, a rendered term may leave the dictionary implicit.

Failure to find one within the bounded scan does not prove the provider unusable in every possible elaboration context. The completeness ledger records that limitation.

29.4 Library recursion as a provider

Suppose a user wants a list transformation that can be expressed by an existing combinator such as `List.map`, `List.foldr`, or a curated recursor. Library mode can make the combinator a premise. Search then synthesizes its arguments rather than inventing recursion.

This decomposition is powerful:

recursive algorithm invention $\implies$ application synthesis over a trusted recursive combinator.

The final term remains ordinary Lean code, and its provider use is checked.

29.5 Why results depend on configuration

Provider and library settings change the search environment, so they change both positive candidates and the meaning of failure. A query that is uninhabited in pure intuitionistic structure may become inhabited after enabling a provider, a classical theorem, or a recursor.

A reproducible synthesis report should therefore include:

- Lean environment/import state;
- provider and library modes;
- engine and resource settings;
- quantifier/recursive policies;
- exact verified candidate.

Chapter checkpoint

Providers bridge imported Lean functionality into the search calculus. Exact names, schemes, type applications, constraints, and environment identity are preserved. Curated recursion converts hard recursive invention into ordinary combinator application. Configuration is part of the query contract.

The Optional Length/Z3 Behavioral Layer

30.1 Why types are not enough

For $\text{List } \alpha \rightarrow \text{List } \alpha$, structural synthesis may return several verified terms. Suppose the user additionally wants output length to equal input length. That property is not encoded in the type, so ordinary Lean verification cannot distinguish identity from constant-empty.

Djex contains an experimental semantic Length layer for a narrowly defined class of list-spine and product problems. Leant can optionally use it to assess already type-correct Exference candidates.

30.2 The authority order

The order is essential:

1. synthesize a candidate through the ordinary engine;
2. render and verify it against the exact Lean type;
3. retain its exact typed candidate graph and inventory sidecar;
4. seal a supported Length contract and target;
5. symbolically interpret the graph under an explicit provider-law basis;
6. encode the resulting bounded arithmetic problem as canonical QF_LIA SMT-LIB;
7. run Z3 through a resource-bounded process/session boundary;
8. independently replay any model or claimed bounded-positive result against the original checked semantic problem;
9. issue only the narrow receipt or ranking decision authorized by that replay.

Z3 never makes an ill-typed term well typed. Lean verification happened first. Conversely, Lean type verification does not prove the Length postcondition; that is a separate, explicitly bounded semantic claim.

30.3 Sealed contracts and typed graphs

The behavioral layer does not reparse the rendered Lean string. It consumes an engine-owned typed term graph tied to:

- the exact synthesis session and inventory;
- provider semantics admitted for Length interpretation;
- the normalized target type and contract;
- the candidate graph fingerprint;
- resource and domain policies.

This prevents a textually identical term from inheriting a receipt produced under another inventory or contract.

30.4 Symbolic length interpretation

Supported term-graph nodes are interpreted as arithmetic expressions over input lengths. A provider such as identity may have law $\ell_{out} = \ell_{in}$; constant empty has $\ell_{out} = 0$; append has $\ell_{out} = \ell_1 + \ell_2$ under a sealed provider law.

Unsupported nodes fail closed. The interpreter does not guess semantics from function names. Provider laws belong to the checked session inventory.

30.5 SMT solving and replay

The arithmetic problem is encoded in quantifier-free linear integer arithmetic. A satisfiable counterexample query may produce concrete natural input lengths violating the postcondition. Djex re-evaluates those inputs through its own checked semantic problem before returning a validated counterexample.

An unsatisfiable solver response is not automatically a proof. Bounded-positive receipts require the exact applicable domain and replay policy recorded by the current API. The semantic documentation repeatedly emphasizes that raw solver status, bytes, or cached samples grant no independent authority.

30.6 What the layer can and cannot say

It can help rank or filter candidates under a supported length contract and bounded domain. It cannot by itself establish:

- extensional equality of arbitrary Lean functions;
- element preservation or permutation;
- correctness outside the sealed domain;
- semantics of unsupported providers;
- a kernel theorem unless a separate proof-producing bridge is built.

Behavioral checks are orthogonal to type checking

A candidate may be type-correct and behaviorally rejected; another may be behaviorally acceptable under a bounded contract but still rely on axioms or providers the user dislikes. Leant keeps these judgments separate so the user can see which property was established by which mechanism.

Source trail

The current semantic-origin and Length boundary is summarized in Leant's `docs/synth-internals.md`. The Djex guide and semantic foundations at the pinned submodule revision describe typed candidates, sealed contracts, QF_LIA queries, bounded Z3 sessions, and independent replay. The standalone Djex main revision contains later refinements that should not be attributed to Leant until its submodule advances.

Chapter checkpoint

The Length layer is post-type-check behavioral analysis over sealed typed candidates. It uses Z3 as a bounded reasoning component and independently replays results. Its receipts have narrow contract-specific authority and are not Lean proofs of general program correctness.

A Reliable Mental Model for Using and Extending Leant

31.1 Ask four questions about every result

When Leant shows a candidate or a failure, ask:

1. **What did Lean elaborate?** Surface notation and auto-implicit variables may hide the exact goal.
2. **What did the fragment expose?** Which pieces became logic, applications, inductives, providers, or opaque atoms?
3. **What did the engine prove or search?** Djinn formula proof, Exference bounded candidate, or both?
4. **What did the final checker authorize?** Lean type acceptance, negative completeness evidence, or a separate behavioral receipt?

These questions prevent most category errors.

31.2 Classify goals before searching

A useful manual classification is:

Pure structural

Arrows, products, sums, polymorphic plumbing. Excellent :synth targets.

Provider structural

Requires a known library function but little semantic reasoning. Enable provider/library search.

Opaque dependent cargo

Dependent formulas are only moved or stored. Often works.

Dependent elimination

Requires instantiation at a value, indexed cases, transport, or a motive. Use Lean tactics/proof mode; synthesis may help later.

Recursive invention

Requires choosing a recursion scheme and branch behavior. Prefer curated combinators or explicit design.

Behaviorally underdetermined

Many inhabitants share the type. Add a theorem, refinement, examples, or a supported semantic contract.

31.3 Extend at the correct layer

New functionality belongs in different places depending on its semantic role:

- recognize another exact Lean connective: serializer/fragment policy;
- support another Haskell-like type form: Djex shared source representation and both engine adapters;
- add a proof-search rule: Djinn calculus, proof terms, and proof checker;

- add a heuristic expression form: Exference nodes, unification, checking, and ranking;
- reconstruct a known engine term in Lean: renderer plus verification tests;
- strengthen negative claims: completeness ledger and exactness proof, not only more search;
- add a behavioral property: typed graph semantics, sealed contract, solver/certificate/replay boundary;
- support general dependent synthesis: a new calculus and term representation, not an extra opaque-atom case.

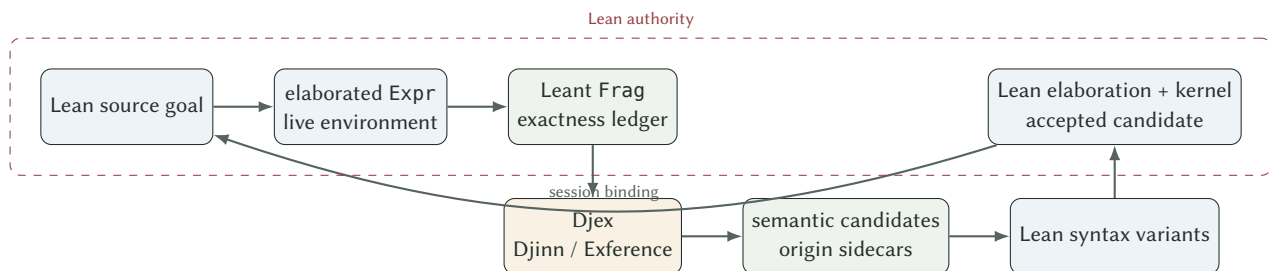
31.4 Prefer fail-closed boundaries

At every translation, ask what happens when metadata is missing or a bound is exceeded. The robust answer is usually:

- keep the subtree opaque for positive transport;
- mark negative evidence unsafe or incomplete;
- reject malformed protocol data;
- drop an unrenderable candidate;
- let Lean reject ambiguous syntax;
- report bounded failure as unknown;
- refuse unsupported behavioral interpretation.

This style may miss answers, but it prevents an optimization or convenience path from acquiring proof authority accidentally.

31.5 The compact architectural picture



The dashed region is not saying the entire Lean implementation is formally verified. It indicates where the final type-authority operation occurs. Djex and the Haskell controller are outside that authority and are made safe for positive answers by the return path through Lean.

Chapter checkpoint

Classify the goal, inspect the fragment, distinguish engine contracts, and identify the final authority. Extend the layer that owns the missing semantics. Preserve fail-closed behavior when exactness cannot be established. This mental model is sufficient to navigate both the codebase and user-facing results.

Conclusion

Lean’s apparent complexity comes from one coherent decision: the type language is expressive enough for types to mention values, propositions to be types, and proofs to be programs. The Calculus of Constructions supplies the dependent function core; inductive families supply data, recursion, equality, and induction; universes and termination keep the system consistent; elaboration lets humans write concise source; the kernel reduces the result to a small checked expression language.

Leant becomes straightforward once that stack is visible. It does not replace Lean’s type theory with Djex. It asks Lean to project a goal into an explicitly smaller structural language. Djinn can decide an exact intuitionistic fragment; Exference can search more flexibly under resource bounds. Unsupported dependent structure is retained as opaque cargo, not erased. Candidate terms return through a renderer that is allowed to be wrong because Lean checks every answer. Negative claims are rarer because they need a complete translation and search ledger. Optional behavioral analysis remains a separate, sealed judgment.

That architecture yields a useful division of labor:

Component	Responsibility
Lean elaborator	interpret concise source in the exact live environment
Lean kernel	authorize typing, universes, conversion, inductives, and safety
Leant fragment	state what structure is exposed and what completeness is lost
Djinn	construct or refute proofs in an exact propositional calculus
Exference	search ranked typed expressions under explicit limits
Leant renderer	reconstruct candidate Lean syntax and bounded variants
Verification loop	return every positive answer to Lean
Behavioral layer	assess a separate sealed property without borrowing type authority

The shortest durable summary is therefore:

Rich source, restricted search, authoritative return

Leant is useful not because Djex understands all of Lean, but because Leant knows exactly when it does not, preserves that boundary as data, and sends every proposed inhabitant back to Lean.

Lean Syntax and Logic Cheat Sheet

A.1 Declarations

Form	Meaning
<code>def f (x : A) : B := t</code>	transparent named definition checked at type B
<code>theorem h : P := p</code>	opaque checked proof of proposition P
<code>example : T := t</code>	check a temporary unnamed declaration
<code>opaque c : T</code>	introduce a checked opaque constant (with a body in full form)
<code>axiom a : T</code>	assume a constant of type T without a body
<code>inductive I ... where ...</code>	declare an inductive family and constructors
<code>structure S ... where ...</code>	one-constructor inductive with named fields
<code>class C ... where ...</code>	structure designated for instance search
<code>instance : C A := ...</code>	register a term for type-class resolution
<code>namespace N ... end N</code>	qualify enclosed names
<code>open N</code>	make namespace members available for resolution/printing
<code>import M</code>	load a compiled module and its environment declarations

A.2 Binders

Syntax	Kind	Caller behavior
<code>(x : A)</code>	explicit	normally supplied
<code>{x : A}</code>	implicit	normally inferred
<code>{{x : A}}</code>	strict implicit	inferred when later explicit use triggers insertion
<code>[x : C]</code>	instance implicit	synthesized by type-class resolution
<code>∀ x : A, B x</code>	dependent binder	proposition/function according to codomain sort

A.3 Term forms

Syntax	Reading
<code>fun x => t</code>	lambda abstraction
<code>f x y</code>	left-associated application
<code>A → B → C</code>	right-associated function type
<code>let x := a; t</code>	local definition
<code>match s with ...</code>	pattern matching, elaborated to recursors
<code>(a, b)</code>	anonymous constructor selected by expected type
<code>p.1, p.2, p.field</code>	projections
<code>_</code>	elaboration metavariable / placeholder
<code>@f</code>	expose normally implicit arguments of f at the surface
<code>f (α := A)</code>	named explicit assignment to an implicit parameter
<code>by ...</code>	tactic script constructing a term
<code>rfl</code>	reflexivity term/tactic, succeeds up to definitional equality

Syntax	Reading
$h \triangleright x$	transport/rewrite x along equality h

A.4 Logical forms

Lean	Constructor/eliminator intuition	Search shape
$P \rightarrow Q$	lambda / application	implication
$P \wedge Q$	pair / projections	conjunction
$P \vee Q$	injection / cases	disjunction
True	unique constructor	top
False	no constructor / eliminate anywhere	bottom
$\neg P$	function $P \rightarrow \text{False}$	implication to bottom
$\forall x : A, P x$	dependent lambda/application	full dependency; only selected fragments exposed
$\exists x : A, P x$	witness + proof / dependent cases	dependent inductive in Prop
$x = y$	Eq.refl / transport	indexed inductive

A.5 Inspection commands useful around Leant

Command	Purpose
#check t	elaborate t and print its type
#eval t	compile/evaluate a computable expression
#reduce t	reduce an expression through the elaborator/kernel reduction facilities
#print name	print a declaration
#print axioms name	report axiomatic dependencies
set_option pp.all true	expose more elaborated detail in pretty printing
set_option trace.Meta.synthInstance true	inspect type-class search (version/options permitting)

Compact Formal Rules

This appendix collects a schematic core sufficient to decode the theoretical discussion. It is not a complete formal specification of Lean's kernel, inductive checker, quotient primitives, safety classes, or optimized conversion algorithm.

B.1 Contexts and sorts

Context extension

$$\frac{\Gamma \text{ ctx} \quad \Gamma \vdash A : \text{Sort } u \quad x \notin \Gamma}{\Gamma, x : A \text{ ctx}}$$

Sort formation

$$\Gamma \vdash \text{Sort } u : \text{Sort}(u + 1).$$

Lean additionally validates that every named universe parameter is declared and that no level metavariable reaches the kernel.

B.2 Dependent products

Π formation

$$\frac{\Gamma \vdash A : \text{Sort } u \quad \Gamma, x : A \vdash B : \text{Sort } v}{\Gamma \vdash \Pi(x : A), B : \text{Sort}(\text{imax}(u, v))}$$

Lambda

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda(x : A), t : \Pi(x : A), B}$$

Application

$$\frac{\Gamma \vdash f : \Pi(x : A), B \quad \Gamma \vdash a : A}{\Gamma \vdash fa : B[a/x]}$$

B.3 Local definitions

Let

$$\frac{\Gamma \vdash A : \text{Sort } u \quad \Gamma \vdash a : A \quad \Gamma, x : A \vdash t : B}{\Gamma \vdash (\text{let } x : A := a; t) : B[a/x]}$$

Zeta reduction makes the let expression definitionally equal to $t[a/x]$.

B.4 Conversion

Definitional conversion

$$\frac{\Gamma \vdash t : A \quad A \equiv B \quad \Gamma \vdash B : \text{Sort } u}{\Gamma \vdash t : B}$$

Lean's implemented $\equiv$ includes reduction and congruence, proof irrelevance, function eta expansion, eta for suitable nonrecursive structures, literal reduction, and environment-guided lazy unfolding.

B.5 Dependent pairs

A schematic Σ rule is:

Σ introduction

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B[a/x]}{\Gamma \vdash \langle a, b \rangle : \Sigma(x : A), B}$$

Lean implements Sigma as an inductive family rather than a primitive Expr constructor. Its eliminator is therefore generated by the inductive machinery.

B.6 Equality elimination

A schematic dependent eliminator is:

$$\text{Eq.rec} : \Pi\{\alpha : \text{Sort } u\}\{a : \alpha\}(P : \Pi b : \alpha, a = b \rightarrow \text{Sort } v), P \text{ a rfl} \rightarrow \Pi\{b : \alpha\}(h : a = b), P b h.$$

Simpler transport is derived by choosing a motive that ignores the equality proof. Pattern matching and rewriting elaborate to applications of such eliminators.

B.7 Inductive declarations

A full rule must check at least:

- constructor types are well formed in declared universes;
- constructor results target the declared inductive family with correct parameters;
- recursive occurrences satisfy positivity;
- universe and elimination constraints are respected;

- mutual blocks are coherent;
- generated recursors type check.

Lean's kernel compiles the accepted block into ordinary constants and recursors. User pattern matching is then elaborated to those constants.

Lean-to-Leant Translation Reference

Lean shape	Leant/Djex treatment	Consequence
non-dependent $A \rightarrow B$	FArr, then function/implication	fully structural when children exact
And P Q	product/conjunction plus nominal restoration	Djinn splits/builds pair
Prod A B, PProd A B	Lean product fragment plus nominal restoration	same logical search shape, exact expected-type check
Or P Q, Sum A B, PSum A B	tagged sum	injections and case analysis
Iff P Q	pair of implications	exact logical expansion
Not P	$P \rightarrow \perp$	exact definitional/logical expansion
true/unit families	top/unit	immediate constructor
false/empty families	nominal bottom	no introduction; elimination if premise exists
sort-bound forall	FAll, rigid Djex forall	selected polymorphism and bounded rank-N use
non-dependent value forall	arrow	ordinary lambda/application
value-dependent forall	opaque atom unless dedicated case	can be moved; not generally instantiated
instance-implicit forall	FInst or exact provider context	positive elaboration possible; negative evidence guarded
type-valued nominal application	FApp retaining head and args	higher-kinded unification/provider matching
term-indexed application	opaque identity	no index-sensitive reasoning
complete nonrecursive	constructor-tagged sum of	construction and case analysis
nonindexed inductive	products	
indexed inductive	opaque	no dependent constructor refinement
constructor with dependent field	opaque	no dependent pair elimination
recursive inductive	bounded recursive fragment	limited positive support; completeness guarded
incomplete declaration	partial fragment/opaque	positive candidates checked; no strong negative
inventory	downgrade	
traversal fuel exhausted	depth marker/completeness downgrade	failure means unknown
concrete unsupported constant	unsafe opaque atom for refutation	may hide a known inhabitant
abstract universally quantified subtree	possibly refutation-safe opaque atom	parametric structural reasoning may remain complete

Interpreting Leant Outcomes

Observed outcome	What is authorized	What is not authorized
Lean-verified candidate	the displayed term inhabits the exact goal in the live environment	intended behavior, efficiency, uniqueness, completeness
Djinn proof before Lean verification	a proof term exists in the translated formula calculus	valid Lean syntax or exact Lean typing
all render variants rejected	this reconstruction route produced no accepted term	source type uninhabited
Exference step limit	no candidate in explored prefix	search exhaustion or uninhabitation
Exference natural exhaustion	no candidate in sealed finite Exference search state	general Lean uninhabitation
Djinn budget exhausted	no proof in explored choice prefix	formula unprovable
unbounded Djinn formula exhaustion	translated exact formula unprovable	Lean goal uninhabited unless completeness ledger also passes
proof-backed Leant uninhabited report	no inhabitant in the exact admitted query contract	no future import/provider could ever inhabit the type
unsafe opaque/completeness warning	positive search remains usable; negative is downgraded	conclusive absence
validated Length counterexample	exact retained candidate violates sealed contract on replayed bounded input	ill-typedness or general semantic failure
bounded-positive Length receipt	no violation under the exact supported domain/contract and replay policy	general Lean theorem outside that contract

Glossary

Alpha-equivalence

Equality up to consistent renaming of bound variables.

Atom

An indivisible proposition/type symbol in a smaller search calculus.

Binder information

Lean metadata marking a binder explicit, implicit, strict implicit, or instance implicit.

Candidate group

One semantic candidate together with bounded alternative Lean renderings.

Calculus of Constructions (CoC)

A dependent lambda calculus supporting all axes of the lambda cube.

Calculus of Inductive Constructions (CIC)

A CoC-family theory extended with inductive families and eliminators.

Completeness

The property that exhaustive search finds a proof whenever one exists in the stated calculus and environment.

Completeness ledger

Leant metadata recording whether translation, inventories, instantiations, and search justify a negative result.

Context

Ordered local declarations under which a term is typed.

Definitional equality

Equality recognized by the type checker through computation and built-in conversion rules.

Dependent function / Π -type

A function type whose codomain may mention the input.

Dependent pair / Σ -type

A pair whose second component type may mention the first component.

Elaboration

Translation from concise Lean syntax to fully explicit expressions using expected types, metavariables, unification, coercions, and instances.

Environment

The checked global map from names to declarations and metadata.

Exference

Djex's heuristic best-first typed expression search engine.

Fragment

Leant's frontend-owned representation of admitted searchable structure plus opacity and completeness metadata.

Impredicative

Permitting quantification over a universe while the result remains in that same universe; Lean's Prop is impredicative.

Indexed family

A type family whose constructors may return different indices, such as `Vector  $\alpha$  n`.

Inhabitant

A term having a given type.

Instance implicit

A binder whose argument is synthesized by Lean's type-class resolver.

LJT

Dyckhoff's contraction-free sequent calculus for intuitionistic propositional logic, used by Djinn.

Metavariable

A temporary elaboration-time unknown with a context and expected type.

Motive

The dependent result family supplied to an inductive recursor or dependent match.

Opaque cargo

A dependent or unsupported type retained as one alpha-aware atom so it can be transported structurally without internal analysis.

Proof irrelevance

Kernel principle identifying all proofs of the same proposition for definitional equality.

Provider

A selected live Lean declaration made available to synthesis as a checked premise.

Recursor

The primitive eliminator generated for an inductive family, supporting recursion and induction.

Refutation safety

Lean's classification of whether hidden fragment structure can be ignored when interpreting exhaustive failure.

Rigid variable

A universally quantified variable that unification may not instantiate.

Semantic origin

Sidecar data tying a candidate to its exact goal, environment, inventory, search policy, and typed provenance.

Soundness

Here, principally the property that every accepted candidate truly has the requested type.

Transport

Converting a term between dependent types using an equality proof.

Universe

A type whose elements are types; Lean represents universes as `Sort u`.

Weak-head normal form

Reduction sufficient to expose a term's outer constructor without fully normalizing every subterm.

Pinned Source Reading Guide

F.1 Leant at 49c8f9f100a6

README.md

User-facing synthesis examples, engine modes, dependent cargo, inductive and negative-result policy.

docs/Leant.tex

Existing LuaLaTeX manual and Unicode-safe transcript style.

docs/synth-internals.md

Semantic-origin, typed-candidate, verification, and behavioral-layer invariants.

src/Leant/Synth/Fragment.hs

Lean-generated serializer, Frag, provider protocol, parser, and candidate verification command.

src/Leant/Synth/Engine.hs

Fragment-to-Djex boundary, search staging, candidate groups, and refutation gate.

src/Leant/Synth/Render.hs

Lean syntax reconstruction, binder restoration, names, and variants.

src/Leant/Synth/Verification.hs

Lazy group verification and accepted-candidate quota.

src/Leant/Backend.hs

Lean REPL process, JSON protocol, timeout, and stderr management.

src/Main.hs

Interactive state, commands, sessions, providers, proof mode, and Length integration.

Repository root: <https://github.com/VladimirReshetnikov/Leant/tree/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0>

F.2 Djex

Leant vendors 7d65eba6f753; the standalone main snapshot inspected for current architecture is 0cf82f2aa22d.

README.md

Engine overview, shared semantic layer, search contracts, and optional Length/Z3 architecture.

synthesis/src/Language/Haskell/Synthesis/Type.hs

Shared Haskell-like type representation and binder/substitution invariants.

synthesis/src/Language/Haskell/Synthesis/Kind.hs

Shared kind tree and ground-kind sealing.

djinn/src-core/Djinn/Internal/LJTFormula.hs

Propositional formulas, opaque source-type atoms, and proof terms.

djinn/src-core/Djinn/Internal/LJT.hs

LJT search, strategies, budgets, proof production, and normalization.

exference/src-core/Language/Haskell/Exference/Core/Internal/Exference.hs

Validated best-first search, options, completion states, and typed results.

Standalone root: <https://github.com/VladimirReshetnikov/Djex/tree/0cf82f2aa22debb1de2c7aaa13a2abbe190550d8>

Vendored revision: <https://github.com/VladimirReshetnikov/Djex/tree/7d65eba6f753727ae7b3d734fb4a7e00f1f44aa7>

F.3 Lean 4 at 79c4cd09fdab

src/Lean/Level.lean

Universe-level syntax and operations.

src/Lean/Expr.lean

Core expression constructors, binder information, locally nameless representation, literals, metadata, and projections.

src/Lean/Declaration.lean

Declaration categories, inductive metadata, recursor compilation, safety, and reducibility hints.

src/kernel/type_checker.cpp

Kernel type inference, Π universe calculation, application substitution, weak-head normalization, definitional equality, proof irrelevance, eta, and declaration safety checks.

Repository root: <https://github.com/leanprover/lean4/tree/79c4cd09fdab845aa6066bcbb09a663876601027>

Bibliography

- [1] Lean Developers, *The Lean Language Reference*, official online reference manual. <https://lean-lang.org/doc/reference/la test/>
- [2] Jeremy Avigad, Leonardo de Moura, Soonho Kong, Sebastian Ullrich, and contributors, *Theorem Proving in Lean 4*, official online textbook. https://lean-lang.org/theorem_proving_in_lean4/
- [3] Leonardo de Moura and Sebastian Ullrich, “The Lean 4 Theorem Prover and Programming Language,” in *Automated Deduction – CADE 28*, LNCS 12699, 2021, pp. 625–635.
- [4] Thierry Coquand and Gérard Huet, “The Calculus of Constructions,” *Information and Computation*, 76(2–3), 1988, pp. 95–120.
- [5] Henk Barendregt, “Introduction to Generalised Type Systems,” *Journal of Functional Programming*, 1(2), 1991, pp. 125–154.
- [6] Roy Dyckhoff, “Contraction-Free Sequent Calculi for Intuitionistic Logic,” *Journal of Symbolic Logic*, 57(3), 1992, pp. 795–807.
- [7] Peter Dybjer, “Inductive Families,” *Formal Aspects of Computing*, 6(4), 1994, pp. 440–465.
- [8] Christine Paulin-Mohring, “Inductive Definitions in the System Coq: Rules and Properties,” in *Typed Lambda Calculi and Applications*, LNCS 664, 1993, pp. 328–345.
- [9] Vladimir Reshetnikov, *Leant*, source repository, pinned at commit 49c8f9f100a6. <https://github.com/VladimirReshetnikov/Leant/tree/49c8f9f100a644b5a32b3c942bc6565a3a3ba6a0>
- [10] Vladimir Reshetnikov, *Djex*, source repository, Leant-vendored revision 7d65eba6f753 and inspected standalone revision 0cf82f2aa22d. <https://github.com/VladimirReshetnikov/Djex>
- [11] Lean Developers, *Lean 4*, source repository, pinned at commit 79c4cd09fdab. <https://github.com/leanprover/lean4/tree/79c4cd09fdab845aa6066bcb09a663876601027>